Bài 1.4 – Pandas: đọc, lọc, nhóm, nối dữ liệu

> **Giai đoạn 1 – Python cho ML & Data Science** · Thời lượng: 4 buổi · Hình thức mở đầu: *So sánh ví von*

Chia buổi:

- **Buổi 1:** Mục 1 → 6.6 (Series, DataFrame, đọc và "khám sức khỏe" dữ liệu).
- **Buổi 2:** Mục 6.7 → 6.13 (chọn, lọc, tạo cột, sửa giá trị, sắp xếp, chia khoảng).
- **Buổi 3:** Mục 6.14 → 6.19 (đếm, nhóm, pivot, nối bảng, dữ liệu chữ, dữ liệu thời gian).
- **Buổi 4:** Mục 6.20 → hết (thiếu dữ liệu, lưu file, viết code gọn và nhanh, lỗi thường gặp, trắc nghiệm, thực hành).

---

1. 🎯 Mục tiêu bài học

Sau bài này bạn có thể...

- Phân biệt **Series**, **DataFrame**, **Index** và giải thích vì sao Pandas "khớp theo nhãn" chứ không theo vị trí.
- Đọc một file dữ liệu thật 17.000 dòng và "khám sức khỏe" nó trong 5 phút: kích thước, kiểu dữ liệu, ô trống, thống kê, giá trị bất thường.
- Chọn, lọc, tạo cột mới, sửa giá trị, sắp xếp bằng `loc`, `iloc`, điều kiện, `np.where`, `map`, `apply`.
- Trả lời câu hỏi kiểu "trung bình theo từng nhóm" bằng `groupby`, `pivot_table`, `crosstab`; xoay bảng bằng `melt`.
- Nối nhiều bảng bằng `merge`/`concat` mà không làm mất hay nhân bản dòng; xử lý cột chữ (`.str`) và cột ngày giờ (`.dt`).

---

2. 🔗 Ôn nhanh kiến thức cần có

- **Dict** (bài 1.1): `{"quan": "Quận 1", "gia": 6.5}`. DataFrame được tạo từ dict rất tự nhiên.
- **Hàm và lambda** (bài 1.2): `lambda x: x * 2`. Dùng trong `apply`, `map`.
- **File CSV** (bài 1.2): mỗi dòng một bản ghi, các cột ngăn bằng dấu phẩy.
- **NumPy** (bài 1.3): mảng, vector hóa, mặt nạ `&` `|` `~`, `np.where`, `axis`, `np.nan`. Mỗi cột Pandas **chính là** một mảng NumPy có gắn nhãn.

---

3. 🧩 Mở đầu: Pandas là "Excel có siêu năng lực"

Chị Lan quản lý dữ liệu cho một chuỗi 30 cửa hàng tiện lợi. Mỗi sáng thứ Hai chị mở file Excel 800.000 dòng, chờ 2 phút để file mở, bấm Filter, kéo công thức, tạo Pivot Table, copy sang file báo cáo. Mất cả buổi sáng. Có tuần máy treo, có tuần chị quên một bước lọc và báo cáo sai.

Nếu bạn từng dùng Excel, bạn đã biết 80% tư duy của Pandas:

Việc trong Excel	Trong Pandas	Ưu điểm của Pandas
Mở file .xlsx/.csv	`pd.read_csv("file.csv")`	Mở file hàng triệu dòng không bị treo

Việc trong Excel	Trong Pandas	Ưu điểm của Pandas
Xem vài dòng đầu	`df.head()`	
Bấm Filter, chọn điều kiện	`df[df["gia"] > 3]`	Lưu lại thành code, chạy lại được
Gõ công thức `=B2/C2` rồi kéo xuống	`df["moi"] = df["B"] / df["C"]`	Không cần kéo, áp dụng cả cột
Hàm IF	`np.where(df["tuoi"] >= 30, "Cũ", "Mới")`	
Pivot Table	`df.pivot_table(...)`, `df.groupby(...)`	Linh hoạt hơn nhiều
VLOOKUP	`pd.merge(bang1, bang2, on="ma")`	Không lỗi khi chèn thêm cột
Sort	`df.sort_values("gia")`	Giống hệt
Find & Replace	`df["cot"].str.replace(...)`	Dùng được quy tắc phức tạp

Khác biệt lớn nhất: trong Excel bạn **bấm chuột** và quên mất mình đã làm gì. Trong Pandas, mọi bước là **code**. Tuần sau có dữ liệu mới, chị Lan chỉ cần bấm "Run all": 10 giây, không sai bước nào.

Và quan trọng nhất với bạn: **mọi dự án ML đều bắt đầu bằng Pandas**. Trước khi dạy máy học, bạn phải đọc, hiểu, lọc, gộp, làm sạch dữ liệu. Công việc này chiếm 60–80% thời gian của người làm ML.

---

4. 💡 Trực giác

4.1. Series là "một cột có số nhà"

Hãy tưởng tượng một dãy phố. Mỗi căn nhà có **số nhà** (nhãn) và **người ở** (giá trị). Bưu tá giao thư theo **số nhà**, không theo "căn thứ mấy tính từ đầu phố". Nếu phố có căn bị phá đi, số nhà của các căn còn lại không đổi.

Series cũng vậy: mỗi giá trị có một **nhãn** gọi là **index**. Pandas làm mọi thứ dựa trên nhãn.

4.2. DataFrame là "một sheet Excel", mỗi cột là một Series

Các cột dùng chung một bộ số nhà (index). Mỗi cột có **một kiểu dữ liệu** riêng: cột giá là số, cột quận là chữ, cột ngày là thời gian. Khác với NumPy (cả mảng chỉ một kiểu), DataFrame cho phép **mỗi cột một kiểu**.

4.3. groupby là "chia bài theo màu"

Bạn có bộ bài 52 lá. Câu hỏi: "Tổng điểm của từng chất (cơ, rô, chuồn, bích)?"

1. **Tách:** chia bài thành 4 chồng theo chất.
2. **Tính:** cộng điểm từng chồng.
3. **Gộp:** ghi 4 kết quả vào một bảng nhỏ.

Đó chính là `df.groupby("chat")["diem"].sum()`.

4.4. merge là "ghép thẻ học sinh với sổ điểm"

Phòng giáo vụ có **sổ điểm** chỉ ghi mã học sinh. Muốn biết tên, phải tra **danh sách lớp** theo mã. Ghép hai tờ giấy theo cột chung "mã học sinh" chính là `merge`.

---

5. 📖 Lý thuyết chi tiết

5.1. Giải phẫu một DataFrame

```
                ┌──────────── columns (tên cột) ────────────┐
                   quan        dien_tich   so_phong   gia_ty
        ┌   0     Quận 1           50         2        6.5     ← một dòng = một
mẫu (row)
  index │   1     Bình Thạnh       65         2        4.2
 (nhãn  │   2     Thủ Đức          80         3        3.8
  dòng) │   3     Quận 7          100         3        7.0
        └   4     Bình Thạnh       72         3        4.9
                    ↑                                  ↑
             cột chữ (object)                  cột số (float64)
             mỗi cột = một Series = một đặc trưng (feature)
```

- **index**: nhãn của dòng. Mặc định là 0, 1, 2... nhưng có thể là chữ, ngày tháng...
- **columns**: nhãn của cột.
- **values**: phần dữ liệu bên trong, thực chất là các mảng NumPy.

5.2. Index và quy tắc "khớp theo nhãn" (alignment)

Khi tính toán giữa hai Series, Pandas **ghép cặp theo nhãn**, không theo vị trí:

```
doanh_thu_T1:  A=100  B=200  C=300
doanh_thu_T2:         B=110  C=190  D=320

T2 - T1 →      A=NaN  B=-90  C=-110 D=NaN
               (A chỉ có ở T1, D chỉ có ở T2 → không có cặp → NaN)
```

Đây là tính năng **rất mạnh** (không sợ lệch dòng khi dữ liệu bị sắp xếp lại) và cũng là **nguồn lỗi bất ngờ** (xuất hiện NaN khi nhãn không khớp). Bạn sẽ thấy cả hai mặt ở mục 6.2 và mục 7.

5.3. Kiểu dữ liệu (dtype) trong Pandas

dtype	Chứa gì	Ví dụ	Ghi chú
`int64`	Số nguyên	Số phòng	Không chứa được NaN (cột có NaN tự thành float)
`float64`	Số thực	Giá, diện tích	NaN là một giá trị float
`object`	Chữ (hoặc trộn lẫn)	Quận, tên	Pandas 3.x hiển thị là `str`

dtype	Chứa gì	Ví dụ	Ghi chú
`bool`	True/False	Có thang máy?	
`category`	Chữ lặp lại ít giá trị	Quận, giới tính	Tiết kiệm bộ nhớ, có thể có thứ tự
`datetime64[ns]`	Ngày giờ	Ngày đăng tin	Dùng được `.dt.month`, so sánh ngày

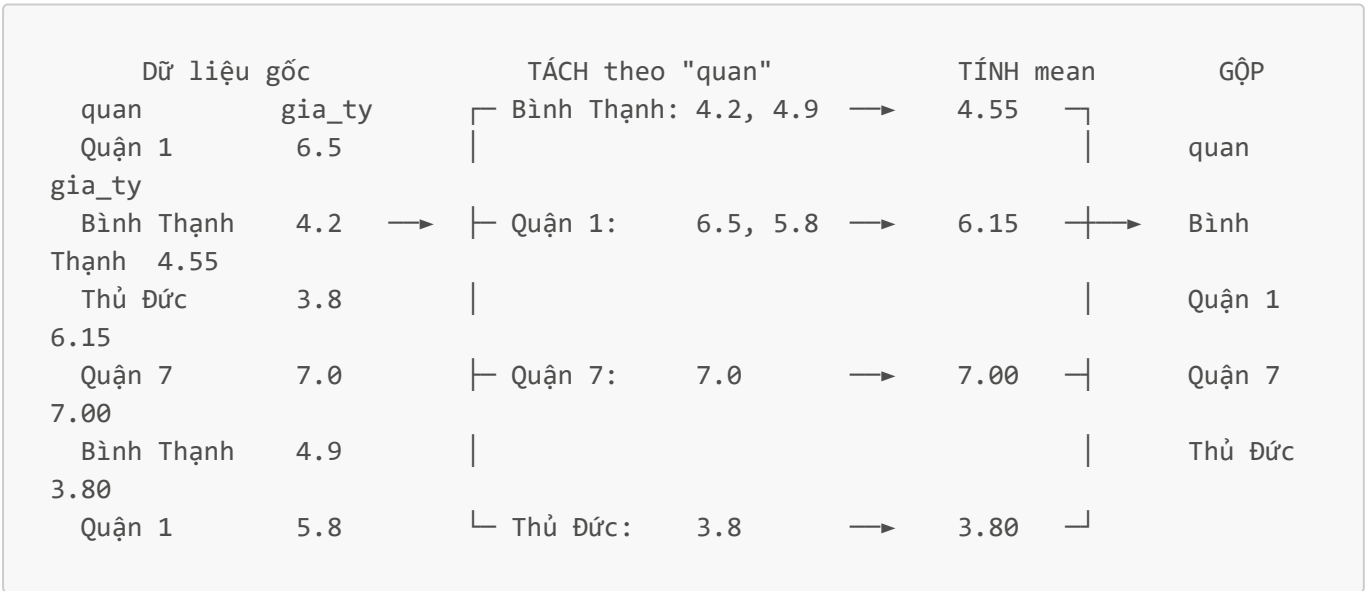
> ⚠️ Cột số mà hiện dtype `object` là **dấu hiệu dữ liệu bẩn**: có ô chứa chữ như `"1,200"` hoặc `"N/A"`. Cộng cột đó sẽ ra kết quả vô nghĩa (mục 7, lỗi 6).

5.4. `axis` trong Pandas

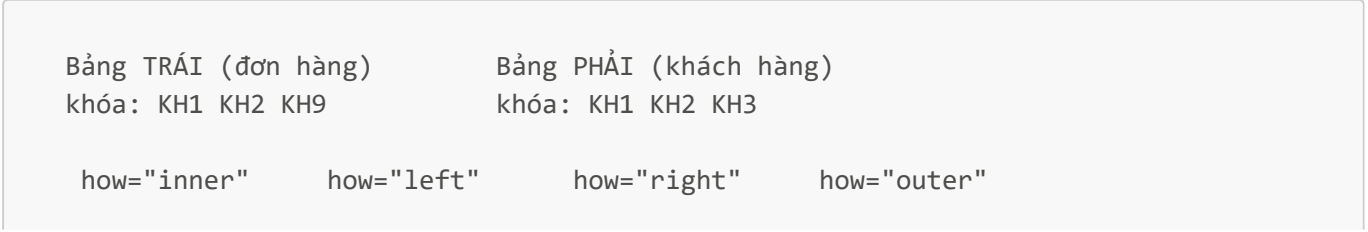
Giống NumPy (bài 1.3): `axis=0` (hoặc `axis="index"`) là dọc theo dòng, `axis=1` (hoặc `axis="columns"`) là dọc theo cột.

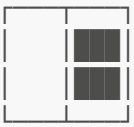
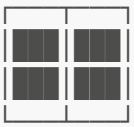
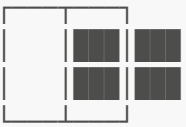
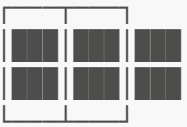
Lệnh	Nghĩa
`df.mean()` / `df.mean(axis=0)`	Trung bình **mỗi cột** (mặc định)
`df.mean(axis=1)`	Trung bình **mỗi dòng**
`df.drop(columns=["a"])`	Xóa cột (rõ ràng hơn `df.drop("a", axis=1)`)
`df.drop(index=[0, 1])`	Xóa dòng

5.5. Split – Apply – Combine (Tách – Tính – Gộp)



5.6. Bốn kiểu nối bảng



```
chỉ KH1, KH2   KH1, KH2, KH9   KH1, KH2, KH3   KH1, KH2, KH3, KH9
               (KH9: tên NaN)  (KH3: tiền NaN)  (thiếu bên nào → NaN bên đó)
```

5.7. 📌 Thuật ngữ

Tên tiếng Anh	Nghĩa tiếng Việt	Giải thích 1 câu	Ví dụ
Pandas (Panel Data)	Thư viện bảng dữ liệu	Thư viện xử lý dữ liệu dạng bảng trong Python.	Đọc CSV, lọc, nhóm
Series	Chuỗi dữ liệu	Một cột dữ liệu có nhãn cho từng giá trị.	Cột giá nhà
DataFrame	Bảng dữ liệu	Bảng nhiều cột, mỗi cột là một Series.	Một sheet Excel
Index	Chỉ mục / nhãn dòng	Tên gọi của mỗi dòng, dùng để tra cứu và khớp dữ liệu.	0, 1, 2 hoặc mã khách hàng
dtype	Kiểu dữ liệu	Kiểu chung của các giá trị trong một cột.	`float64`, `object`
NaN (Not a Number)	Giá trị thiếu	Ký hiệu ô trống / không có dữ liệu.	Khách không khai tuổi
Boolean indexing	Lọc bằng điều kiện	Chọn dòng bằng một cột True/False.	`df[df.gia > 3]`
groupby	Nhóm dữ liệu	Tách bảng theo nhóm, tính riêng từng nhóm, gộp kết quả.	Giá TB theo quận
Aggregation (agg)	Tổng hợp	Biến nhiều giá trị thành một con số.	`mean`, `sum`, `count`
Pivot table	Bảng tổng hợp chéo	Bảng có dòng là nhóm A, cột là nhóm B, ô là con số tổng hợp.	Giá TB theo quận × số phòng
merge / join	Nối bảng	Ghép hai bảng theo cột chung.	VLOOKUP
concat	Nối chồng	Chồng các bảng lên nhau (thêm dòng) hoặc đặt cạnh nhau.	Gộp dữ liệu 12 tháng

Tên tiếng Anh	Nghĩa tiếng Việt	Giải thích 1 câu	Ví dụ
Wide / Long format	Dạng rộng / dạng dài	Rộng: mỗi tháng một cột. Dài: một cột "tháng", một cột "giá trị".	`melt`, `pivot`
Method chaining	Viết nối chuỗi	Nối nhiều thao tác liên tiếp bằng dấu chấm.	`df.query(...).groupby(...).mean()`

6. 💻 Code từng bước (Colab)

> Chạy các ô **theo đúng thứ tự**. Kết quả mẫu trong bài được tạo với pandas 2.2. Nếu Colab của bạn dùng pandas 3.x, cột chữ sẽ hiện dtype `str` thay vì `object`; mọi thứ khác giống nhau.

6.1. Import

```
import pandas as pd       # quy ước đặt tên tắt là pd
import numpy as np        # Pandas thường đi cùng NumPy
print(pd.__version__)     # ví dụ: 2.2.2
```

6.2. Series: một cột có nhãn

6.2.1. Tạo Series

```
# Từ list: index mặc định 0, 1, 2...
s = pd.Series([2.5, 3.1, 4.0])
print(s)

# Từ list + index tự đặt
gia = pd.Series([2.5, 3.1, 4.0], index=["Căn A", "Căn B", "Căn C"], name="gia_ty")
print(gia)

# Từ dict: khóa thành index, giá trị thành dữ liệu
dan_so = pd.Series({"Hà Nội": 8.6, "TP.HCM": 9.5, "Đà Nẵng": 1.2})   # triệu người
print(dan_so)
```

Kết quả mẫu:

```
0    2.5
1    3.1
2    4.0
dtype: float64
Căn A    2.5
Căn B    3.1
Căn C    4.0
```

```
Name: gia_ty, dtype: float64
Hà Nội     8.6
TP.HCM     9.5
Đà Nẵng    1.2
dtype: float64
```

Dòng cuối `dtype: float64` cho biết kiểu dữ liệu. `Name: gia_ty` là tên của Series, sẽ thành tên cột khi ghép vào DataFrame.

6.2.2. Lấy giá trị

```
print(gia["Căn B"])            # theo NHÃN
print(gia.iloc[0])             # theo VỊ TRÍ (iloc = integer location)
print(gia[["Căn A", "Căn C"]]) # nhiều nhãn → Series con
print(gia.index.tolist(), gia.values)   # danh sách nhãn, mảng NumPy giá trị
```

Kết quả mẫu:

```
3.1
2.5
Căn A    2.5
Căn C    4.0
Name: gia_ty, dtype: float64
['Căn A', 'Căn B', 'Căn C'] [2.5 3.1 4. ]
```

6.2.3. Tính toán và thống kê

```
print(gia * 1000)                      # vector hóa như NumPy, giữ nguyên nhãn
print(gia[gia > 3])                    # lọc bằng điều kiện, giữ nguyên nhãn
print(round(gia.mean(), 2), gia.max(), gia.idxmax())   # idxmax: NHÃN của giá trị
lớn nhất (NumPy chỉ có argmax = vị trí)
print(gia.describe())                  # bộ thống kê nhanh
```

Kết quả mẫu:

```
Căn A    2500.0
Căn B    3100.0
Căn C    4000.0
Name: gia_ty, dtype: float64
Căn B    3.1
Căn C    4.0
Name: gia_ty, dtype: float64
3.2 4.0 Căn C
count    3.000000
```

```
mean     3.200000
std      0.754983
min      2.500000
25%      2.800000
50%      3.100000
75%      3.550000
max      4.000000
Name: gia_ty, dtype: float64
```

6.2.4. Khớp theo nhãn (alignment)

```
thang_1 = pd.Series([100, 200, 300], index=["A", "B", "C"])   # doanh thu tháng 1 của 3 cửa hàng
thang_2 = pd.Series([190, 320, 110], index=["C", "D", "B"])   # tháng 2: thứ tự khác, có cửa hàng mới D, A đóng cửa

print(thang_2 - thang_1)                       # Pandas ghép theo NHÃN, không theo vị trí
print(thang_2.sub(thang_1, fill_value=0))      # thiếu bên nào coi như 0
```

Kết quả mẫu:

```
A      NaN
B    -90.0
C   -110.0
D      NaN
dtype: float64
A   -100.0
B    -90.0
C   -110.0
D    320.0
dtype: float64
```

- Dù `thang_2` viết theo thứ tự C, D, B, Pandas vẫn lấy đúng B tháng 2 trừ B tháng 1. Với NumPy, bạn phải tự sắp xếp cho khớp.
- A và D không có cặp nên ra `NaN`. Kết quả tự đổi sang `float64` vì NaN là số thực.
- `sub(..., fill_value=0)`: phép trừ có điền giá trị thiếu. Tương tự có `add`, `mul`, `div`.

6.3. DataFrame: bảng dữ liệu

6.3.1. Ba cách tạo DataFrame

```
# Cách 1 (hay dùng nhất): dict, mỗi khóa là TÊN CỘT, mỗi giá trị là LIST dữ liệu của cột đó
nha = pd.DataFrame({
    "quan": ["Quận 1", "Bình Thạnh", "Thủ Đức", "Quận 7", "Bình Thạnh", "Quận 1"],
```

```
    "dien_tich": [50, 65, 80, 100, 72, 45],
    "so_phong": [2, 2, 3, 3, 3, 1],
    "gia_ty": [6.5, 4.2, 3.8, 7.0, 4.9, 5.8],
})
print(nha)

# Cách 2: list các dict, mỗi dict là MỘT DÒNG (giống dữ liệu JSON từ API trả về)
ban_ghi = [
    {"ten": "An", "tuoi": 25, "thanh_pho": "Hà Nội"},
    {"ten": "Bình", "tuoi": 31},                        # thiếu thành phố → NaN
]
print(pd.DataFrame(ban_ghi))

# Cách 3: từ mảng NumPy 2 chiều + tên cột
mang = np.array([[1, 2], [3, 4]])
print(pd.DataFrame(mang, columns=["x1", "x2"]))
```

Kết quả mẫu:

```
         quan  dien_tich  so_phong  gia_ty
0      Quận 1         50         2     6.5
1  Bình Thạnh         65         2     4.2
2     Thủ Đức         80         3     3.8
3      Quận 7        100         3     7.0
4  Bình Thạnh         72         3     4.9
5      Quận 1         45         1     5.8
    ten  tuoi thanh_pho
0    An    25    Hà Nội
1  Bình    31       NaN
   x1  x2
0   1   2
1   3   4
```

6.3.2. Thuộc tính cơ bản

```
print(nha.shape)              # (số dòng, số cột)
print(nha.columns.tolist())   # danh sách tên cột
print(nha.index)              # nhãn dòng: RangeIndex là dãy 0..5
print(nha.dtypes)             # kiểu dữ liệu từng cột
```

Kết quả mẫu:

```
(6, 4)
['quan', 'dien_tich', 'so_phong', 'gia_ty']
RangeIndex(start=0, stop=6, step=1)
quan          object
dien_tich      int64
```

```
so_phong       int64
gia_ty       float64
dtype: object
```

6.4. Đọc dữ liệu thật

6.4.1. Giới thiệu dataset: Giá nhà California

Ta dùng dữ liệu điều tra dân số California (Mỹ) năm 1990, bản rút gọn trong khóa Machine Learning Crash Course của Google. Đây là dataset "nhập môn" kinh điển của ML.

> ⚠️ **Đọc kỹ trước khi dùng:** mỗi dòng **không phải** một căn nhà, mà là một **khu dân cư** (vài trăm đến vài nghìn người). Giá nhà là **giá trung vị** của khu đó, đơn vị **USD**. Thu nhập là thu nhập trung vị, đơn vị **chục nghìn USD/năm** (ví dụ 3.5 = 35.000 USD). `total_rooms` là **tổng** số phòng của cả khu, không phải của một căn.

Hiểu sai ý nghĩa cột là lỗi phổ biến và nguy hiểm nhất khi phân tích dữ liệu.

6.4.2. `read_csv`

```
# Đọc thẳng từ internet. Trên Colab cũng có sẵn file này ở:
# "/content/sample_data/california_housing_train.csv"
url = "https://download.mlcc.google.com/mledu-
datasets/california_housing_train.csv"
df = pd.read_csv(url)        # df là tên quy ước cho DataFrame chính
print(df.shape)              # (số dòng, số cột)
```

Kết quả mẫu:

```
(17000, 9)
```

Một số tham số hay dùng của `read_csv`:

```
nho = pd.read_csv(url, nrows=3, usecols=["median_income", "median_house_value"])
# chỉ đọc 3 dòng, 2 cột
print(nho)
```

Kết quả mẫu:

```
   median_income  median_house_value
0         1.4936             66900.0
1         1.8200             80100.0
2         1.6509             85700.0
```

Tham số	Ý nghĩa	Khi nào dùng
`sep=";"`	Ký tự ngăn cột	File xuất từ Excel tiếng Việt/châu Âu hay dùng `;`
`encoding="utf-8"`	Bảng mã chữ	Lỗi font tiếng Việt thì thử `"utf-8-sig"` hoặc `"cp1258"`
`usecols=[...]`	Chỉ đọc vài cột	File quá lớn
`nrows=1000`	Chỉ đọc n dòng đầu	Xem thử file khổng lồ
`parse_dates=["ngay"]`	Đổi cột sang kiểu ngày	Dữ liệu có thời gian (mục 6.19)
`index_col="ma"`	Dùng một cột làm index	Có cột mã định danh
`na_values=["N/A", "-"]`	Coi các ký hiệu này là NaN	Dữ liệu ghi "thiếu" bằng chữ

Các định dạng khác cũng có hàm tương tự: `pd.read_excel("file.xlsx")`, `pd.read_json(...)`, `pd.read_parquet(...)`, `pd.read_sql(...)`. Ghi ra thì dùng `df.to_csv`, `df.to_excel`...

6.4.3. Đổi tên cột sang tiếng Việt cho dễ nhớ

```
df = df.rename(columns={                # dict {tên cũ: tên mới}
    "longitude": "kinh_do", "latitude": "vi_do",
    "housing_median_age": "tuoi_nha", "total_rooms": "tong_phong",
    "total_bedrooms": "tong_phong_ngu", "population": "dan_so",
    "households": "so_ho", "median_income": "thu_nhap",
    "median_house_value": "gia_nha",
})
pd.set_option("display.max_columns", None)   # hiện đủ mọi cột khi in
pd.set_option("display.width", 200)          # cho phép in bảng rộng trên một dòng
print(df.head())                              # xem 5 dòng đầu
```

Kết quả mẫu:

```
   kinh_do  vi_do  tuoi_nha  tong_phong  tong_phong_ngu  dan_so  so_ho  thu_nhap
gia_nha
0  -114.31  34.19      15.0      5612.0          1283.0  1015.0  472.0    1.4936
66900.0
1  -114.47  34.40      19.0      7650.0          1901.0  1129.0  463.0    1.8200
80100.0
2  -114.56  33.69      17.0       720.0           174.0   333.0  117.0    1.6509
85700.0
3  -114.57  33.64      14.0      1501.0           337.0   515.0  226.0    3.1917
73400.0
4  -114.57  33.57      20.0      1454.0           326.0   624.0  262.0    1.9250
65500.0
```

> 💡 Trên Colab, nếu dòng cuối của ô là `df.head()` (không có `print`), Colab sẽ hiển thị một bảng HTML đẹp hơn. Cả hai cách đều đúng.

Quy tắc đặt tên cột tốt: chữ thường, không dấu, không khoảng trắng, nối bằng `_`. Khi đó bạn gõ code ít lỗi hơn.

6.5. "Khám sức khỏe" dataset

Mọi dự án nên chạy 5 lệnh này ngay sau khi đọc dữ liệu.

6.5.1. `head`, `tail`, `sample`

```
print(df.tail(3))                       # 3 dòng cuối
print(df.sample(3, random_state=42))    # 3 dòng ngẫu nhiên; random_state = seed
(bài 1.3)
```

Kết quả mẫu:

```
       kinh_do  vi_do  tuoi_nha  tong_phong  tong_phong_ngu  dan_so  so_ho
thu_nhap   gia_nha
16997  -124.30  41.84      17.0      2677.0           531.0  1244.0  456.0
3.0313  103600.0
16998  -124.30  41.80      19.0      2672.0           552.0  1298.0  478.0
1.9797   85800.0
16999  -124.35  40.54      52.0      1820.0           300.0   806.0  270.0
3.0147   94600.0
       kinh_do  vi_do  tuoi_nha  tong_phong  tong_phong_ngu  dan_so  so_ho
thu_nhap   gia_nha
10941  -120.87  37.77       9.0      4838.0           920.0  2460.0  923.0
3.5959  142700.0
5250   -118.14  34.11      52.0      2742.0           422.0  1153.0  414.0
8.1124  500001.0
10292  -120.05  36.98      16.0      3705.0           739.0  2463.0  697.0
2.5288   61800.0
```

Vì sao dùng `sample`? Nhiều file được sắp xếp sẵn (file này xếp theo kinh độ). `head()` chỉ cho bạn thấy góc đông nam của bang California, có thể khiến bạn hiểu sai cả bộ dữ liệu.

6.5.2. `info()`: kiểu dữ liệu và ô trống

```
df.info()
```

Kết quả mẫu:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17000 entries, 0 to 16999
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   kinh_do         17000 non-null  float64
 1   vi_do           17000 non-null  float64
 2   tuoi_nha        17000 non-null  float64
 3   tong_phong      17000 non-null  float64
 4   tong_phong_ngu  17000 non-null  float64
 5   dan_so          17000 non-null  float64
 6   so_ho           17000 non-null  float64
 7   thu_nhap        17000 non-null  float64
 8   gia_nha         17000 non-null  float64
dtypes: float64(9)
memory usage: 1.2 MB
```

Đọc: 17.000 dòng, mọi cột đều có đủ 17.000 giá trị (**non-null** = không trống). Tất cả là số thực. Dataset này khá sạch. Bài 1.6 sẽ cho bạn gặp dữ liệu "bẩn".

6.5.3. `describe()`: thống kê nhanh mọi cột số

```
print(df.describe().round(2))     # round(2): làm tròn 2 số lẻ cho dễ đọc
```

Kết quả mẫu:

```
        kinh_do     vi_do  tuoi_nha  tong_phong  tong_phong_ngu    dan_so
so_ho  thu_nhap    gia_nha
count  17000.00  17000.00  17000.00    17000.00        17000.00  17000.00
17000.00  17000.00   17000.00
mean    -119.56     35.63     28.59     2643.66          539.41   1429.57
501.22      3.88  207300.91
std        2.01      2.14     12.59     2179.95          421.50   1147.85
384.52      1.91  115983.76
min     -124.35     32.54      1.00        2.00            1.00      3.00
1.00      0.50   14999.00
25%     -121.79     33.93     18.00     1462.00          297.00    790.00
282.00      2.57  119400.00
50%     -118.49     34.25     29.00     2127.00          434.00   1167.00
409.00      3.54  180400.00
75%     -118.00     37.72     37.00     3151.25          648.25   1721.00
605.25      4.77  265000.00
max     -114.31     41.95     52.00    37937.00         6445.00  35682.00
6082.00     15.00  500001.00
```

Dòng	Ý nghĩa	(sẽ học kỹ ở)
count	Số ô có dữ liệu	
mean	Trung bình	bài 1.7
std	Độ lệch chuẩn: dữ liệu phân tán rộng hay hẹp	bài 1.7
min / max	Nhỏ nhất / lớn nhất	
25% / 50% / 75%	Phân vị. 50% chính là **trung vị**	bài 1.7

Thám tử dữ liệu 🔍: Nhìn cột `gia_nha`, bạn có thấy gì lạ không?

- `max` = **500001**, một con số "tròn" đáng ngờ.
- Đó là vì người thu thập đã **giới hạn trần**: mọi khu có giá trên 500.000 USD đều bị ghi là 500.001. Ta sẽ đếm xem có bao nhiêu khu như vậy ở mục 6.14.
- `tuoi_nha` max = 52, `thu_nhap` max = 15: cũng có thể bị giới hạn trần. Ta sẽ kiểm chứng.
- `tong_phong` max = 37.937 trong khi 75% số khu dưới 3.152 → có vài khu **cực lớn** (outlier).

Đây là kỹ năng quan trọng nhất của người làm ML: **nghi ngờ dữ liệu trước khi tin nó**.

6.5.4. Đếm giá trị khác nhau và ô trống

```
print(df.nunique())               # mỗi cột có bao nhiêu giá trị KHÁC NHAU
print(df.isna().sum().sum())      # tổng số ô trống của cả bảng
```

Kết quả mẫu:

```
kinh_do             827
vi_do               840
tuoi_nha             52
tong_phong         5533
tong_phong_ngu     1848
dan_so             3683
so_ho              1740
thu_nhap          11175
gia_nha            3694
dtype: int64
0
```

`tuoi_nha` chỉ có 52 giá trị khác nhau trên 17.000 dòng: đây là số nguyên năm, từ 1 đến 52. Trong khi `thu_nhap` có hàng nghìn giá trị: số thực liên tục. Biết điều này giúp bạn chọn biểu đồ (bài 1.5) và cách mã hóa (bài 1.6).

6.6. Bài tập nhanh cuối buổi 1 (5 phút)

Không nhìn lại phía trên, trả lời:

1. Lệnh nào cho biết cột nào có ô trống?
2. Vì sao nên dùng `sample` thay vì chỉ `head`?
3. Cột `gia_nha` có điều gì đáng ngờ?

(Đáp án: 1. `info()` hoặc `isna().sum()`. 2. Vì file có thể đã được sắp xếp. 3. Bị giới hạn trần ở 500.001.)

6.7. Chọn cột

```
print(df["gia_nha"].head(3))                # 1 cột → ra Series
print(type(df["gia_nha"]))
print(df[["thu_nhap", "gia_nha"]].head(3))  # nhiều cột → truyền LIST tên cột → ra DataFrame
```

Kết quả mẫu:

```
0    66900.0
1    80100.0
2    85700.0
Name: gia_nha, dtype: float64
<class 'pandas.core.series.Series'>
   thu_nhap  gia_nha
0    1.4936  66900.0
1    1.8200  80100.0
2    1.6509  85700.0
```

Để ý **hai lớp ngoặc** `[[...]]` khi chọn nhiều cột: ngoặc ngoài là "lấy từ df", ngoặc trong là "danh sách tên cột".

> 💡 Bạn sẽ thấy cách viết `df.gia_nha` (dấu chấm). Nó chạy được nhưng **không dùng được** khi tên cột có khoảng trắng, có dấu, hoặc trùng tên method (ví dụ cột tên `count`, `mean`). Và **không dùng được để tạo cột mới**. Hãy quen dùng `df["ten_cot"]`.

6.8. Chọn dòng: `iloc`, `loc`, `at`

6.8.1. So sánh `iloc` và `loc`

	`iloc`	`loc`
Chọn theo	**Vị trí** (số thứ tự 0, 1, 2...)	**Nhãn** (tên index, tên cột)
Mẹo nhớ	**i** = **i**nteger (số nguyên)	**l** = **l**abel (nhãn)
Cắt lát `0:2`	Lấy 0, 1 (bỏ 2)	Lấy 0, 1, **2** (tính cả 2)
Chọn bằng điều kiện	Không	**Có**: `df.loc[mask, "cot"]`

```
print(df.iloc[0])                           # dòng ở vị trí 0 (ra Series)
print(df.iloc[0:3, 0:2])                    # dòng 0–2, cột 0–1 (theo vị trí,
```

```
bỏ biên phải)
print(df.loc[0:2, ["tuoi_nha", "gia_nha"]])    # dòng có nhãn 0 ĐẾN 2 (tính cả 2),
cột theo tên
print(df.iloc[-1, -1])                         # ô ở dòng cuối, cột cuối
```

Kết quả mẫu:

```
kinh_do            -114.3100
vi_do                34.1900
tuoi_nha             15.0000
tong_phong         5612.0000
tong_phong_ngu     1283.0000
dan_so             1015.0000
so_ho               472.0000
thu_nhap              1.4936
gia_nha           66900.0000
Name: 0, dtype: float64
   kinh_do  vi_do
0  -114.31  34.19
1  -114.47  34.40
2  -114.56  33.69
   tuoi_nha  gia_nha
0      15.0  66900.0
1      19.0  80100.0
2      17.0  85700.0
94600.0
```

6.8.2. Khi index không phải 0, 1, 2: thấy rõ khác biệt

```
nha_q = nha.set_index("quan")      # dùng cột "quan" làm index
print(nha_q)
print(nha_q.loc["Thủ Đức"])        # theo nhãn: dòng có index "Thủ Đức"
print(nha_q.loc["Bình Thạnh"])     # nhãn trùng → trả về NHIỀU dòng
print(nha_q.iloc[2])               # theo vị trí: dòng thứ 3, cũng là Thủ Đức
print(nha_q.reset_index().head(2)) # reset_index: đưa index về lại thành cột,
index thành 0, 1, 2...
```

Kết quả mẫu:

```
            dien_tich  so_phong  gia_ty
quan
Quận 1             50         2     6.5
Bình Thạnh         65         2     4.2
Thủ Đức            80         3     3.8
Quận 7            100         3     7.0
Bình Thạnh         72         3     4.9
Quận 1             45         1     5.8
```

```
dien_tich    80.0
so_phong      3.0
gia_ty        3.8
Name: Thủ Đức, dtype: float64
            dien_tich  so_phong  gia_ty
quan
Bình Thạnh         65         2     4.2
Bình Thạnh         72         3     4.9
dien_tich    80.0
so_phong      3.0
gia_ty        3.8
Name: Thủ Đức, dtype: float64
         quan  dien_tich  so_phong  gia_ty
0      Quận 1         50         2     6.5
1  Bình Thạnh         65         2     4.2
```

6.8.3. Lấy một ô: `at`, `iat`

```
print(nha.at[3, "gia_ty"])     # nhãn dòng 3, cột gia_ty – nhanh hơn loc khi chỉ
lấy 1 ô
print(nha.iat[3, 3])           # vị trí dòng 3, cột 3
```

Kết quả mẫu:

```
7.0
7.0
```

6.9. Lọc dữ liệu

6.9.1. Một điều kiện

Giống hệt boolean mask của NumPy (bài 1.3).

```
mask = df["gia_nha"] > 400000          # Series True/False dài 17.000
print(mask.head(3))
dat = df[mask]                         # chỉ giữ dòng True
print(len(dat), "khu có giá trên 400.000 USD")
print(round(mask.mean() * 100, 1), "% số khu")   # tỉ lệ = trung bình của
True/False
```

Kết quả mẫu:

```
0    False
1    False
```

```
2    False
Name: gia_nha, dtype: bool
1458 khu có giá trên 400.000 USD
8.6 % số khu
```

6.9.2. Nhiều điều kiện

```
# Khu thu nhập cao (> 80.000 USD) VÀ nhà mới (< 10 năm). Nhớ NGOẶC và dấu &
loc2 = df[(df["thu_nhap"] > 8) & (df["tuoi_nha"] < 10)]
print(len(loc2))

# Nhà rất cũ HOẶC rất rẻ
print(len(df[(df["tuoi_nha"] >= 50) | (df["gia_nha"] < 50000)]))

# KHÔNG phải khu bị giới hạn trần
print(len(df[~(df["gia_nha"] >= 500000)]))
```

Kết quả mẫu:

```
61
1355
16164
```

6.9.3. `isin`, `between`

```
# Lọc theo danh sách giá trị (thay cho nhiều điều kiện | nối nhau)
print(nha[nha["quan"].isin(["Quận 1", "Quận 7"])])

# Lọc theo khoảng, tính cả hai đầu
print(len(df[df["tuoi_nha"].between(20, 30)]))
```

Kết quả mẫu:

```
     quan  dien_tich  so_phong  gia_ty
0  Quận 1         50         2     6.5
3  Quận 7        100         3     7.0
5  Quận 1         45         1     5.8
4369
```

6.9.4. `query`: viết điều kiện như câu nói

```
print(df.query("thu_nhap > 8 and tuoi_nha < 10").shape)    # dùng and/or bình
thường
nguong = 450000
print(df.query("gia_nha > @nguong").shape)                 # @ để dùng biến Python
bên ngoài
```

Kết quả mẫu:

```
(61, 9)
(1058, 9)
```

6.9.5. Lọc dòng và chọn cột cùng lúc với `loc`

```
# Thu nhập và giá của 3 khu đầu tiên có tuổi nhà = 52
print(df.loc[df["tuoi_nha"] == 52, ["thu_nhap", "gia_nha"]].head(3))
```

Kết quả mẫu:

```
      thu_nhap   gia_nha
325     3.9028  128600.0
1011    4.2813  270800.0
1074    4.3875  300000.0
```

Mẫu `df.loc[điều_kiện, cột]` là cách viết **chuẩn nhất**: vừa lọc, vừa chọn cột, và dùng được để **gán giá trị** (mục 6.11).

6.10. Tạo cột mới

6.10.1. Phép toán trên cả cột

```
# Số phòng trung bình mỗi hộ: chia cả cột cho cả cột (vector hóa)
df["phong_moi_ho"] = df["tong_phong"] / df["so_ho"]

# Số người mỗi hộ
df["nguoi_moi_ho"] = df["dan_so"] / df["so_ho"]

# Đổi giá sang triệu VND (giả sử 1 USD = 25.000 VND)
df["gia_trieu_vnd"] = df["gia_nha"] * 25000 / 1e6     # 1e6 = 1.000.000

print(df[["phong_moi_ho", "nguoi_moi_ho", "gia_trieu_vnd"]].head(3).round(2))
```

Kết quả mẫu:

```
   phong_moi_ho  nguoi_moi_ho  gia_trieu_vnd
0         11.89          2.15         1672.5
1         16.52          2.44         2002.5
2          6.15          2.85         2142.5
```

Liên hệ ML: `phong_moi_ho` là một **đặc trưng mới tạo ra** từ 2 cột có sẵn. Tổng số phòng của cả khu không nói lên nhiều (khu to thì nhiều phòng), nhưng "số phòng mỗi hộ" phản ánh nhà rộng hay chật. Việc tạo đặc trưng thông minh như vậy gọi là **Feature Engineering (Kỹ thuật đặc trưng)** (sẽ học kỹ ở bài 2.16).

6.10.2. Cột theo điều kiện: `np.where` (2 nhóm) và `np.select` (nhiều nhóm)

```
# 2 nhóm: nhà từ 30 năm tuổi là "Cũ", còn lại là "Mới" – giống hàm IF trong Excel
df["loai_nha"] = np.where(df["tuoi_nha"] >= 30, "Cũ", "Mới")

# Nhiều nhóm: liệt kê điều kiện theo thứ tự ưu tiên, kèm nhãn tương ứng
dieu_kien = [df["thu_nhap"] < 3, df["thu_nhap"] < 6]
nhan = ["Thấp", "Trung bình"]
df["muc_thu_nhap"] = np.select(dieu_kien, nhan, default="Cao")   # không thỏa điều
kiện nào → "Cao"

print(df[["tuoi_nha", "loai_nha", "thu_nhap", "muc_thu_nhap"]].head(4))
```

Kết quả mẫu:

```
   tuoi_nha loai_nha  thu_nhap muc_thu_nhap
0      15.0      Mới    1.4936         Thấp
1      19.0      Mới    1.8200         Thấp
2      17.0      Mới    1.6509         Thấp
3      14.0      Mới    3.1917   Trung bình
```

`np.select` xét điều kiện **từ trên xuống**, dừng ở điều kiện đúng đầu tiên. Vì thế `thu_nhap = 4` không thỏa `< 3` nhưng thỏa `< 6` → "Trung bình".

6.10.3. `map`: đổi giá trị theo bảng tra

```
ma_quan = {"Quận 1": "Q1", "Bình Thạnh": "BT", "Thủ Đức": "TĐ"}   # cố tình QUÊN
"Quận 7"
nha["ma_quan"] = nha["quan"].map(ma_quan)          # tra từng giá trị trong dict
print(nha)
print(nha["ma_quan"].isna().sum(), "dòng không tra được")
```

Kết quả mẫu:

```
         quan  dien_tich  so_phong  gia_ty ma_quan
0      Quận 1         50         2     6.5      Q1
1  Bình Thạnh         65         2     4.2      BT
2     Thủ Đức         80         3     3.8      TĐ
3      Quận 7        100         3     7.0     NaN
4  Bình Thạnh         72         3     4.9      BT
5      Quận 1         45         1     5.8      Q1
1 dòng không tra được
```

> ⚠️ Giá trị không có trong dict sẽ thành `NaN` (Quận 7 ở trên). Đây là lỗi hay gặp: viết thiếu một quận trong dict thì cả loạt dòng thành NaN mà **không báo lỗi**. Luôn kiểm tra `isna().sum()` sau khi `map`.

6.10.4. `apply`: áp dụng hàm của bạn

```python
def xep_hang_nha(gia):                    # hàm thường (bài 1.2)
    if gia < 150000:
        return "Bình dân"
    elif gia < 300000:
        return "Trung cấp"
    return "Cao cấp"

df["phan_khuc"] = df["gia_nha"].apply(xep_hang_nha)    # gọi hàm cho TỪNG giá trị
print(df["phan_khuc"].value_counts())

# apply với lambda, theo TỪNG DÒNG (axis=1): dùng được nhiều cột cùng lúc
nha["gia_m2"] = nha.apply(lambda dong: dong["gia_ty"] * 1000 / dong["dien_tich"],
axis=1)
print(nha[["quan", "gia_m2"]].round(1))
```

Kết quả mẫu:

```
phan_khuc
Trung cấp    7568
Bình dân     6217
Cao cấp      3215
Name: count, dtype: int64
         quan  gia_m2
0      Quận 1   130.0
1  Bình Thạnh    64.6
2     Thủ Đức    47.5
3      Quận 7    70.0
4  Bình Thạnh    68.1
5      Quận 1   128.9
```

> ⚠️ `apply` chậm hơn phép toán vector hóa vì nó gọi hàm Python cho từng dòng (mục 6.23 sẽ đo). Ví dụ `gia_m2` ở trên viết vector hóa là `nha["gia_ty"] * 1000 / nha["dien_tich"]`, nhanh hơn rất

> nhiều. Chỉ dùng `apply` khi logic phức tạp, không viết được bằng phép toán cả cột.

6.10.5. Đổi kiểu, xóa cột

```
nha["so_phong"] = nha["so_phong"].astype(float)    # đổi kiểu cột
nha["quan"] = nha["quan"].astype("category")       # category: tiết kiệm bộ nhớ
cho chữ lặp lại
print(nha.dtypes)

nha = nha.drop(columns=["ma_quan"])                # xóa cột. PHẢI gán lại vào
nha!
print(nha.columns.tolist())
```

Kết quả mẫu:

```
quan         category
dien_tich       int64
so_phong      float64
gia_ty        float64
ma_quan        object
gia_m2        float64
dtype: object
['quan', 'dien_tich', 'so_phong', 'gia_ty', 'gia_m2']
```

Hầu hết method của Pandas **trả về bảng mới**, không sửa bảng cũ. Viết `nha.drop(columns=[...])` mà không gán `nha = ...` thì không có gì thay đổi (mục 7, lỗi 5).

6.11. Sửa giá trị trong bảng

```
bang = nha.copy()                                  # làm trên bản sao để giữ dữ
liệu gốc
bang.loc[bang["dien_tich"] < 50, "gia_ty"] = np.nan # đánh dấu căn quá nhỏ là
"chưa rõ giá"
bang.loc[2, "so_phong"] = 4                        # sửa đúng 1 ô: dòng nhãn 2,
cột so_phong
print(bang.round(1))
```

Kết quả mẫu:

```
         quan  dien_tich  so_phong  gia_ty  gia_m2
0      Quận 1         50       2.0     6.5   130.0
1  Bình Thạnh         65       2.0     4.2    64.6
2     Thủ Đức         80       4.0     3.8    47.5
3      Quận 7        100       3.0     7.0    70.0
```

```
4  Bình Thạnh         72       3.0     4.9    68.1
5      Quận 1         45       1.0     NaN   128.9
```

Quy tắc vàng khi sửa: dùng **một** lần `loc[điều_kiện, cột] = giá_trị`.

> ⚠️ **SettingWithCopyWarning.** Hai cách viết sau trông hợp lý nhưng **sai**:
>
> - `bang[bang["dien_tich"] < 50]["gia_ty"] = 0` (hai lần ngoặc liên tiếp, gọi là *chained assignment*): sửa vào một bản sao tạm thời rồi vứt đi, bảng gốc **không đổi**.
> - `con = bang[bang["quan"] == "Quận 1"]` rồi `con["gia_ty"] = 0`: Pandas 2.x in cảnh báo `SettingWithCopyWarning` vì không rõ bạn muốn sửa bảng con hay bảng gốc.
>
> **Cách sửa:** sửa bảng gốc thì dùng `bang.loc[điều_kiện, "gia_ty"] = 0`. Muốn làm việc riêng với bảng con thì tách bằng `.copy()`: `con = bang[bang["quan"] == "Quận 1"].copy()`.

6.12. Sắp xếp và xếp hạng

```
# 3 khu giá cao nhất
print(df.sort_values("gia_nha", ascending=False)[["thu_nhap", "gia_nha"]].head(3))

# Sắp xếp theo NHIỀU cột: tuổi nhà giảm dần, cùng tuổi thì giá tăng dần
print(df.sort_values(["tuoi_nha", "gia_nha"], ascending=[False, True])
[["tuoi_nha", "gia_nha"]].head(3))

# Cách viết gọn: nlargest / nsmallest lấy n dòng lớn/nhỏ nhất
print(df.nlargest(3, "thu_nhap")[["thu_nhap", "gia_nha"]])

# Xếp hạng: căn đắt nhất hạng 1
nha["hang_gia"] = nha["gia_ty"].rank(ascending=False).astype(int)
print(nha.sort_values("hang_gia")[["quan", "gia_ty", "hang_gia"]])
```

Kết quả mẫu:

```
      thu_nhap   gia_nha
2698   10.1122  500001.0
7156    6.8870  500001.0
7157   11.6677  500001.0
       tuoi_nha  gia_nha
3226       52.0  14999.0
11653      52.0  22500.0
6592       52.0  38800.0
      thu_nhap   gia_nha
1625   15.0001  500001.0
3338   15.0001  500001.0
4461   15.0001  500001.0
         quan  gia_ty  hang_gia
3      Quận 7     7.0         1
0      Quận 1     6.5         2
5      Quận 1     5.8         3
```

```
4  Bình Thạnh     4.9         4
1  Bình Thạnh     4.2         5
2     Thủ Đức     3.8         6
```

Lại gặp con số 500001. Và thu nhập cao nhất là **15.0001**, cũng tròn đáng ngờ: cột thu nhập cũng bị giới hạn trần!

`sort_index()` sắp xếp theo index, hay dùng sau khi lọc hoặc nhóm.

6.13. Chia số thành khoảng: `pd.cut` và `pd.qcut`

```
# cut: TỰ ĐẶT ranh giới các khoảng
df["nhom_tuoi"] = pd.cut(df["tuoi_nha"], bins=[0, 10, 20, 30, 40, 52],
                         labels=["0-10", "11-20", "21-30", "31-40", "41-52"])
print(df["nhom_tuoi"].value_counts(sort=False))   # sort=False: giữ thứ tự các
khoảng

# qcut: chia thành các nhóm có SỐ LƯỢNG BẰNG NHAU (theo phân vị)
df["nhom_thu_nhap"] = pd.qcut(df["thu_nhap"], q=4, labels=["Q1", "Q2", "Q3",
"Q4"])
print(df["nhom_thu_nhap"].value_counts(sort=False))
print(df.groupby("nhom_thu_nhap", observed=True)["thu_nhap"].agg(["min",
"max"]).round(2))
```

Kết quả mẫu:

```
nhom_tuoi
0-10     1313
11-20    3896
21-30    3986
31-40    4635
41-52    3170
Name: count, dtype: int64
nhom_thu_nhap
Q1    4250
Q2    4251
Q3    4249
Q4    4250
Name: count, dtype: int64
                min    max
nhom_thu_nhap
Q1             0.50   2.57
Q2             2.57   3.54
Q3             3.55   4.77
Q4             4.77  15.00
```

	pd.cut	pd.qcut
Ranh giới	Bạn tự đặt	Pandas tự tính theo phân vị
Số lượng mỗi nhóm	Có thể chênh lệch nhiều	Gần bằng nhau
Ví dụ	Nhóm tuổi 0–18, 19–35, 36–60	Chia khách hàng thành 4 nhóm chi tiêu bằng nhau

`observed=True` trong `groupby`: chỉ hiện những nhóm thực sự có dữ liệu (bắt buộc ghi với cột kiểu category để không bị cảnh báo).

6.14. Đếm: `value_counts`, `unique`

```
print(df["loai_nha"].value_counts())                          # đếm số lượng
print(df["loai_nha"].value_counts(normalize=True).round(3))   # tỉ lệ thay vì số lượng
print(nha["quan"].unique().tolist(), nha["quan"].nunique())   # các giá trị khác nhau, số lượng

print((df["gia_nha"] >= 500000).sum())                        # số khu bị giới hạn trần giá
print(df["gia_nha"].value_counts().head())                    # 5 giá xuất hiện nhiều nhất
```

Kết quả mẫu:

```
loai_nha
Mới    8811
Cũ     8189
Name: count, dtype: int64
loai_nha
Mới    0.518
Cũ     0.482
Name: proportion, dtype: float64
['Quận 1', 'Bình Thạnh', 'Thủ Đức', 'Quận 7'] 4
836
gia_nha
500001.0    814
137500.0     95
162500.0     89
112500.0     85
187500.0     74
Name: count, dtype: int64
```

- 836 khu (gần 5%) có giá ≥ 500.000, trong đó 814 khu bị ghi đúng 500.001 do giới hạn trần. Khi dạy mô hình dự đoán giá (bài 2.2), mô hình sẽ "tưởng" giá không bao giờ vượt quá 500.001. Đây là vấn đề cần ghi chú lại.

- Các giá "tròn" như 137.500, 162.500, 112.500 xuất hiện bất thường nhiều. Có thể người khai đã làm tròn. Bạn sẽ **thấy** chúng trên biểu đồ ở bài 1.5.

6.15. groupby: Tách – Tính – Gộp

6.15.1. Cơ bản

```
# Giá trung bình theo quận (dữ liệu nhỏ, dễ kiểm tra bằng tay với hình ở mục 5.5)
print(nha.groupby("quan", observed=True)["gia_ty"].mean())

# Giá nhà trung bình của từng loại nhà (dữ liệu California)
print(df.groupby("loai_nha")["gia_nha"].mean().round(0))
```

Kết quả mẫu:

```
quan
Bình Thạnh    4.55
Quận 1        6.15
Quận 7        7.00
Thủ Đức       3.80
Name: gia_ty, dtype: float64
loai_nha
Cũ     215706.0
Mới    199490.0
Name: gia_nha, dtype: float64
```

Điều thú vị: khu nhà cũ lại đắt hơn, dù thu nhập ở đó thấp hơn (ta sẽ thấy ngay dưới đây)! Vì sao? Có thể nhà cũ thường nằm ở trung tâm thành phố lớn (xây từ sớm, đất đắt), còn nhà mới ở vùng ven. Đây chỉ là **giả thuyết**. Bài 1.5 sẽ vẽ bản đồ để kiểm tra.

6.15.2. Nhiều phép tính một lúc: `agg`

```
# Cách 1: list các phép tính cho 1 cột
print(df.groupby("loai_nha")["gia_nha"].agg(["count", "mean", "median",
"max"]).round(0))

# Cách 2 (khuyên dùng): đặt tên cột kết quả. tên_mới=(cột_gốc, phép_tính)
print(df.groupby("loai_nha").agg(
    so_khu=("gia_nha", "count"),
    gia_tb=("gia_nha", "mean"),
    thu_nhap_tb=("thu_nhap", "mean"),
    tuoi_max=("tuoi_nha", "max"),
).round(2))
```

Kết quả mẫu:

```
           count      mean    median       max
loai_nha
Cũ          8189  215706.0  185000.0  500001.0
Mới         8811  199490.0  175000.0  500001.0
           so_khu     gia_tb  thu_nhap_tb  tuoi_max
loai_nha
Cũ           8189  215705.51         3.70      52.0
Mới          8811  199489.63         4.06      29.0
```

Các phép tính hay dùng: `count`, `sum`, `mean`, `median`, `min`, `max`, `std`, `nunique`, `first`, `last`. Bạn cũng có thể truyền hàm của mình, ví dụ `("gia_nha", lambda x: x.max() - x.min())`.

6.15.3. Nhóm theo nhiều cột

```
ket_qua = df.groupby(["muc_thu_nhap", "loai_nha"])["gia_nha"].mean().round(0)
print(ket_qua)                      # index 2 tầng (MultiIndex)
print(ket_qua.reset_index())        # đưa về bảng phẳng, dễ đọc và dễ xử lý tiếp
print(ket_qua.unstack())            # xoay tầng index trong cùng thành cột
```

Kết quả mẫu:

```
muc_thu_nhap  loai_nha
Cao           Cũ          429547.0
              Mới         350798.0
Thấp          Cũ          137720.0
              Mới         128305.0
Trung bình    Cũ          237608.0
              Mới         202218.0
Name: gia_nha, dtype: float64
  muc_thu_nhap loai_nha   gia_nha
0          Cao       Cũ  429547.0
1          Cao      Mới  350798.0
2         Thấp       Cũ  137720.0
3         Thấp      Mới  128305.0
4   Trung bình       Cũ  237608.0
5   Trung bình      Mới  202218.0
loai_nha            Cũ       Mới
muc_thu_nhap
Cao           429547.0  350798.0
Thấp          137720.0  128305.0
Trung bình    237608.0  202218.0
```

Đọc bảng `unstack`: ở **cùng mức thu nhập**, nhà Cũ vẫn đắt hơn nhà Mới. Giả thuyết "vị trí" càng có lý.

6.15.4. `size` khác `count` thế nào?

```
t = pd.DataFrame({"nhom": ["A", "A", "B"], "diem": [5, np.nan, 7]})
print(t.groupby("nhom").size())             # đếm SỐ DÒNG của mỗi nhóm (kể cả dòng
có NaN)
print(t.groupby("nhom")["diem"].count())    # đếm số ô CÓ DỮ LIỆU (bỏ NaN)
```

Kết quả mẫu:

```
nhom
A    2
B    1
dtype: int64
nhom
A    1
B    1
Name: diem, dtype: int64
```

6.15.5. `transform`: so sánh mỗi dòng với nhóm của nó

Câu hỏi: khu này đắt hay rẻ **so với các khu cùng mức thu nhập**?

```
# transform trả về kết quả CÙNG ĐỘ DÀI với bảng gốc (17.000 dòng), không thu gọn
như agg
df["gia_tb_nhom"] = df.groupby("muc_thu_nhap")["gia_nha"].transform("mean")
df["chenh_lech"] = df["gia_nha"] - df["gia_tb_nhom"]
print(df[["muc_thu_nhap", "gia_nha", "gia_tb_nhom",
"chenh_lech"]].head(4).round(0))
```

Kết quả mẫu:

```
  muc_thu_nhap  gia_nha  gia_tb_nhom  chenh_lech
0         Thấp  66900.0     133297.0    -66397.0
1         Thấp  80100.0     133297.0    -53197.0
2         Thấp  85700.0     133297.0    -47597.0
3   Trung bình  73400.0     218984.0   -145584.0
```

```
agg       : 17.000 dòng  ⟶  3 dòng (mỗi nhóm 1 dòng)
transform : 17.000 dòng  ⟶  17.000 dòng (mỗi dòng nhận giá trị của nhóm nó thuộc
về)
```

`transform` rất hay dùng để tạo đặc trưng kiểu "giá so với trung bình khu vực", "chi tiêu so với trung bình khách cùng hạng".

6.16. Bảng chéo: `pivot_table`, `crosstab`, `melt`

6.16.1. `pivot_table`: Pivot Table của Excel

```
print(df.pivot_table(values="gia_nha",        # con số đặt trong ô
                     index="muc_thu_nhap",    # dòng
                     columns="loai_nha",      # cột
                     aggfunc="mean",          # phép tính
                     margins=True,            # thêm dòng/cột tổng "All"
                     ).round(0))
```

Kết quả mẫu:

```
loai_nha            Cũ       Mới       All
muc_thu_nhap
Cao           429547.0  350798.0  379918.0
Thấp          137720.0  128305.0  133297.0
Trung bình    237608.0  202218.0  218984.0
All           215706.0  199490.0  207301.0
```

6.16.2. `crosstab`: đếm theo 2 tiêu chí

```
print(pd.crosstab(df["muc_thu_nhap"], df["loai_nha"]))
print(pd.crosstab(df["muc_thu_nhap"], df["loai_nha"], normalize="index").round(2))
# tỉ lệ theo từng DÒNG
```

Kết quả mẫu:

```
loai_nha        Cũ   Mới
muc_thu_nhap
Cao             734  1251
Thấp           3206  2840
Trung bình     4249  4720
loai_nha        Cũ   Mới
muc_thu_nhap
Cao            0.37  0.63
Thấp           0.53  0.47
Trung bình     0.47  0.53
```

Đọc dòng "Cao" của bảng tỉ lệ: trong các khu thu nhập cao, 63% là nhà Mới. Khu giàu thường là khu mới xây.

6.16.3. `melt` và `pivot`: xoay bảng rộng ↔ dài

Dữ liệu doanh thu từ kế toán thường ở **dạng rộng** (mỗi ngày một cột). Nhưng để nhóm, lọc, vẽ biểu đồ, ta cần **dạng dài**.

```
rong = pd.DataFrame({
    "chi_nhanh": ["Quận 1", "Quận 3"],
    "T2": [12, 9], "T3": [15, 10], "T4": [14, 15],
})
print(rong)

dai = rong.melt(id_vars="chi_nhanh", var_name="ngay", value_name="doanh_thu")   # rộng → dài
print(dai)

print(dai.pivot(index="chi_nhanh", columns="ngay", values="doanh_thu"))       # dài → rộng
```

Kết quả mẫu:

```
  chi_nhanh  T2  T3  T4
0    Quận 1  12  15  14
1    Quận 3   9  10  15
  chi_nhanh ngay  doanh_thu
0    Quận 1   T2         12
1    Quận 3   T2          9
2    Quận 1   T3         15
3    Quận 3   T3         10
4    Quận 1   T4         14
5    Quận 3   T4         15
ngay       T2  T3  T4
chi_nhanh
Quận 1     12  15  14
Quận 3      9  10  15
```

```
DẠNG RỘNG                          DẠNG DÀI
chi_nhanh  T2  T3  T4              chi_nhanh  ngay  doanh_thu
Quận 1     12  15  14    melt ►    Quận 1     T2    12
Quận 3      9  10  15   ◄ pivot    Quận 3     T2     9
                                   Quận 1     T3    15
                                   ...
```

Seaborn (bài 1.5) gần như luôn cần dữ liệu **dạng dài**.

6.17. Nối bảng

6.17.1. Bài toán

Một shop online có 2 bảng: **đơn hàng** (chỉ có mã khách) và **khách hàng** (có tên, thành phố). Muốn biết "đơn nào của ai, ở đâu" thì phải nối lại. Đây chính là VLOOKUP trong Excel, hoặc JOIN trong SQL.

```
don_hang = pd.DataFrame({
    "ma_don": [101, 102, 103, 104],
    "ma_kh": ["KH1", "KH2", "KH1", "KH9"],     # KH9 không có trong bảng khách
hàng!
    "tien": [120000, 85000, 60000, 200000],
})
khach_hang = pd.DataFrame({
    "ma_kh": ["KH1", "KH2", "KH3"],
    "ten": ["An", "Bình", "Chi"],
    "thanh_pho": ["Hà Nội", "TP.HCM", "Đà Nẵng"],
})

print(pd.merge(don_hang, khach_hang, on="ma_kh", how="inner"))   # chỉ giữ dòng
khớp ở CẢ HAI bảng
print(pd.merge(don_hang, khach_hang, on="ma_kh", how="left"))    # giữ MỌI dòng
bảng trái
```

Kết quả mẫu:

```
   ma_don ma_kh    tien   ten thanh_pho
0     101   KH1  120000    An    Hà Nội
1     102   KH2   85000  Bình    TP.HCM
2     103   KH1   60000    An    Hà Nội
   ma_don ma_kh    tien   ten thanh_pho
0     101   KH1  120000    An    Hà Nội
1     102   KH2   85000  Bình    TP.HCM
2     103   KH1   60000    An    Hà Nội
3     104   KH9  200000   NaN       NaN
```

- Đơn 104 trị giá 200.000đ **biến mất** khi dùng inner. Nếu bạn tính tổng doanh thu sau khi inner-merge, bạn sẽ báo cáo thiếu tiền. **Luôn kiểm tra số dòng trước và sau khi merge!**
- `how="left"` là kiểu dùng nhiều nhất: "giữ nguyên bảng chính, tra thêm thông tin".

6.17.2. `outer` và `indicator`: tìm dòng không khớp

```
kq = pd.merge(don_hang, khach_hang, on="ma_kh", how="outer", indicator=True)   #
indicator: thêm cột _merge
print(kq)
print(kq["_merge"].value_counts())
```

Kết quả mẫu:

```
   ma_don ma_kh      tien   ten thanh_pho      _merge
0   101.0   KH1  120000.0    An    Hà Nội        both
1   103.0   KH1   60000.0    An    Hà Nội        both
2   102.0   KH2   85000.0  Bình    TP.HCM        both
3     NaN   KH3       NaN   Chi   Đà Nẵng  right_only
4   104.0   KH9  200000.0   NaN       NaN   left_only
_merge
both          3
left_only     1
right_only    1
Name: count, dtype: int64
```

Cột `_merge` cho biết dòng đến từ đâu: `both` (cả hai), `left_only` (chỉ bảng trái: đơn của khách không tồn tại, có thể lỗi dữ liệu), `right_only` (khách chưa từng mua: đối tượng để gửi khuyến mãi!).

6.17.3. Tên cột khóa khác nhau

```
san_pham = pd.DataFrame({"id": ["SP1", "SP2"], "ten_sp": ["Áo thun", "Quần
jean"]})
chi_tiet = pd.DataFrame({"ma_sp": ["SP1", "SP2", "SP1"], "so_luong": [2, 1, 5]})
print(pd.merge(chi_tiet, san_pham, left_on="ma_sp", right_on="id", how="left"))
```

Kết quả mẫu:

```
  ma_sp  so_luong   id     ten_sp
0   SP1         2  SP1    Áo thun
1   SP2         1  SP2  Quần jean
2   SP1         5  SP1    Áo thun
```

6.17.4. `concat`: chồng bảng

```
khach_moi = pd.DataFrame({"ma_kh": ["KH4"], "ten": ["Dũng"], "thanh_pho":
["Huế"]})
print(pd.concat([khach_hang, khach_moi], ignore_index=True))   # chồng DỌC;
ignore_index: đánh số lại 0, 1, 2...

# Ghép NGANG (axis=1): đặt hai bảng cạnh nhau theo index
diem_toan = pd.DataFrame({"toan": [8, 7]}, index=["An", "Bình"])
diem_van = pd.DataFrame({"van": [6, 9]}, index=["An", "Bình"])
print(pd.concat([diem_toan, diem_van], axis=1))
```

Kết quả mẫu:

```
  ma_kh   ten thanh_pho
0   KH1    An    Hà Nội
1   KH2  Bình    TP.HCM
2   KH3   Chi   Đà Nẵng
3   KH4  Dũng       Huế
      toan  van
An       8    6
Bình     7    9
```

Ứng dụng thật: công ty có 12 file doanh thu theo tháng. Đọc từng file vào một list rồi `pd.concat(danh_sach, ignore_index=True)` là có cả năm.

6.18. Dữ liệu chữ: `.str`

Bình luận khách hàng của một quán ăn trên app giao đồ ăn:

```
bl = pd.DataFrame({
    "khach": ["  an nguyen", "BÌNH TRẦN", "chi lê  ", "Dũng Phạm"],
    "binh_luan": ["Ngon, giao nhanh!", "Giao CHẬM quá, đồ nguội", "Ngon nhưng hơi
mặn", "giao chậm, nhưng ngon"],
    "sdt": ["090-123-4567", "091.234.5678", "0912 345 678", "0987654321"],
})

bl["khach"] = bl["khach"].str.strip().str.title()           # bỏ khoảng trắng
thừa, viết hoa chữ cái đầu
bl["sdt"] = bl["sdt"].str.replace(r"[^0-9]", "", regex=True) # bỏ mọi ký tự KHÔNG
phải số
bl["cham"] = bl["binh_luan"].str.lower().str.contains("chậm")   # có chữ "chậm"?
(đổi chữ thường trước)
bl["ngon"] = bl["binh_luan"].str.contains("ngon", case=False)   # case=False:
không phân biệt hoa thường
bl["so_ky_tu"] = bl["binh_luan"].str.len()
bl["ho"] = bl["khach"].str.split().str[-1]                   # tách theo khoảng
trắng, lấy từ cuối
print(bl)
print("Tỉ lệ phàn nàn giao chậm:", bl["cham"].mean())
```

Kết quả mẫu:

```
       khach                binh_luan         sdt   cham   ngon  so_ky_tu      ho
0  An Nguyen        Ngon, giao nhanh!  0901234567  False   True        17  Nguyen
1  Bình Trần  Giao CHẬM quá, đồ nguội  0912345678   True  False        23    Trần
2     Chi Lê       Ngon nhưng hơi mặn  0912345678  False   True        18      Lê
3  Dũng Phạm    giao chậm, nhưng ngon  0987654321   True   True        21    Phạm
Tỉ lệ phàn nàn giao chậm: 0.5
```

- `.str` cho phép gọi các method của chuỗi (bài 1.1) trên **cả cột** một lúc.

- `r"[^0-9]"` là một **regex (biểu thức chính quy)**, nghĩa là "mọi ký tự không phải chữ số". Bạn chưa cần học regex bây giờ, chỉ cần biết nó tồn tại (sẽ gặp lại ở bài NLP1.2).
- Đếm từ khóa như "chậm", "ngon" là bước đầu tiên của **phân tích cảm xúc bình luận**, bài toán chính của nhánh NLP.

> 💡 Ở dữ liệu thật, cột `ho` lấy từ cuối chỉ đúng khi tên ghi theo kiểu Tây (tên trước, họ sau). Tên Việt "Nguyễn Văn An" thì từ đầu mới là họ. Luôn kiểm tra quy ước của dữ liệu.

6.19. Dữ liệu thời gian: `to_datetime` và `.dt`

Đơn hàng của một quán cà phê trong 2 tuần:

```
ban_hang = pd.DataFrame({
    "ngay": ["2026-03-02", "2026-03-03", "2026-03-05", "2026-03-07", "2026-03-08",
             "2026-03-09", "2026-03-11", "2026-03-14", "2026-03-14", "2026-03-
15"],
    "mon": ["Cà phê sữa", "Trà đào", "Cà phê sữa", "Bạc xỉu", "Trà đào",
            "Cà phê sữa", "Bạc xỉu", "Cà phê sữa", "Trà đào", "Bạc xỉu"],
    "so_ly": [30, 12, 28, 40, 35, 31, 18, 45, 38, 42],
})
print(ban_hang.dtypes)                                    # "ngay" đang là CHỮ
(object)
ban_hang["ngay"] = pd.to_datetime(ban_hang["ngay"])       # đổi sang kiểu ngày
giờ
print(ban_hang.dtypes)
```

Kết quả mẫu:

```
ngay     object
mon      object
so_ly     int64
dtype: object
ngay     datetime64[ns]
mon              object
so_ly             int64
dtype: object
```

Khi đã là kiểu ngày, bạn lấy được mọi thông tin với `.dt`:

```
ban_hang["thu"] = ban_hang["ngay"].dt.dayofweek        # 0 = Thứ Hai, ..., 6 = Chủ
Nhật
ban_hang["cuoi_tuan"] = ban_hang["thu"] >= 5           # Thứ Bảy, Chủ Nhật
ban_hang["tuan"] = ban_hang["ngay"].dt.isocalendar().week   # số thứ tự tuần trong
năm
print(ban_hang.head(5))

print(ban_hang.groupby("cuoi_tuan")["so_ly"].mean())   # cuối tuần bán nhiều hơn
không?
```

```
print(ban_hang[ban_hang["ngay"] >= "2026-03-10"])      # so sánh ngày bằng chuỗi
"YYYY-MM-DD"
print(ban_hang["ngay"].max() - ban_hang["ngay"].min()) # trừ hai ngày → khoảng
thời gian
```

Kết quả mẫu:

```
        ngay         mon  so_ly  thu  cuoi_tuan  tuan
0 2026-03-02  Cà phê sữa     30    0      False    10
1 2026-03-03     Trà đào     12    1      False    10
2 2026-03-05  Cà phê sữa     28    3      False    10
3 2026-03-07     Bạc xỉu     40    5       True    10
4 2026-03-08     Trà đào     35    6       True    10
cuoi_tuan
False    23.8
True     40.0
Name: so_ly, dtype: float64
        ngay         mon  so_ly  thu  cuoi_tuan  tuan
6 2026-03-11     Bạc xỉu     18    2      False    11
7 2026-03-14  Cà phê sữa     45    5       True    11
8 2026-03-14     Trà đào     38    5       True    11
9 2026-03-15     Bạc xỉu     42    6       True    11
13 days 00:00:00
```

`resample`: gộp theo chu kỳ thời gian (ngày, tuần, tháng):

```
theo_tuan = ban_hang.set_index("ngay")["so_ly"].resample("W").sum()   # "W" = tuần
(kết thúc Chủ Nhật)
print(theo_tuan)
```

Kết quả mẫu:

```
ngay
2026-03-08    145
2026-03-15    174
Freq: W-SUN, Name: so_ly, dtype: int64
```

Với dữ liệu thời gian, `resample("D")` (ngày), `"W"` (tuần), `"MS"` (tháng, nhãn là đầu tháng) là công cụ chính để làm báo cáo và dự báo doanh số.

6.20. Dữ liệu thiếu và trùng lặp (giới thiệu, học kỹ ở bài 1.6)

```
khach = pd.DataFrame({
    "ten": ["An", "Bình", "Chi", "Bình", "Em"],
    "tuoi": [25, np.nan, 31, np.nan, 28],
```

```
    "thanh_pho": ["Hà Nội", "Huế", None, "Huế", "Đà Nẵng"],
})
print(khach.isna().sum())                    # đếm ô trống mỗi cột
print(khach.dropna())                        # bỏ MỌI dòng có ít nhất một ô trống
print(khach.fillna({"tuoi": khach["tuoi"].median(), "thanh_pho": "Không rõ"}))   #
điền mỗi cột một kiểu
print(khach.duplicated().sum())              # có bao nhiêu dòng TRÙNG HẲN dòng
phía trên
print(khach.drop_duplicates())               # bỏ dòng trùng
```

Kết quả mẫu:

```
ten          0
tuoi         2
thanh_pho    1
dtype: int64
  ten  tuoi thanh_pho
0  An  25.0    Hà Nội
4  Em  28.0   Đà Nẵng
    ten  tuoi thanh_pho
0    An  25.0    Hà Nội
1  Bình  28.0       Huế
2   Chi  31.0  Không rõ
3  Bình  28.0       Huế
4    Em  28.0   Đà Nẵng
1
    ten  tuoi thanh_pho
0    An  25.0    Hà Nội
1  Bình   NaN       Huế
2   Chi  31.0      None
4    Em  28.0   Đà Nẵng
```

Xóa hay điền? Điền bằng gì? Đây là quyết định quan trọng, ảnh hưởng trực tiếp đến mô hình. Bài 1.6 sẽ dạy cách chọn.

6.21. Lưu kết quả

```
df.to_csv("nha_da_xu_ly.csv", index=False)   # index=False: không ghi cột số thứ
tự thừa
doc_lai = pd.read_csv("nha_da_xu_ly.csv")
print(doc_lai.shape)                         # đọc lại để kiểm tra
print(doc_lai.columns.tolist()[9:])          # các cột mới tạo trong bài
```

Kết quả mẫu:

```
(17000, 19)
['phong_moi_ho', 'nguoi_moi_ho', 'gia_trieu_vnd', 'loai_nha', 'muc_thu_nhap',
```

```
 'phan_khuc', 'nhom_tuoi', 'nhom_thu_nhap', 'gia_tb_nhom', 'chenh_lech']
```

- Lưu CSV làm **mất kiểu dữ liệu** đặc biệt: cột category, datetime đọc lại sẽ thành chữ. Muốn giữ nguyên, dùng `df.to_parquet("file.parquet")` (nhanh và nhẹ hơn CSV nhiều lần với dữ liệu lớn).
- Xuất Excel cho sếp: `df.to_excel("bao_cao.xlsx", index=False)`.
- Trên Colab, nhớ lưu vào Google Drive (bài 0.2, mục 4.8) nếu muốn giữ file.

6.22. Method chaining: viết cả quy trình trong một câu lệnh

Câu hỏi: "Trong các khu nhà **không bị giới hạn trần giá**, nhóm thu nhập nào có giá **mỗi phòng** cao nhất?"

```
ket_qua = (
    df
    .query("gia_nha < 500000")                                    # 1. bỏ khu bị
trần
    .assign(gia_moi_phong=lambda d: d["gia_nha"] / d["tong_phong"] * d["so_ho"])
# 2. tạo cột mới
    .groupby("nhom_thu_nhap", observed=True)                      # 3. nhóm
    .agg(so_khu=("gia_nha", "size"), gia_moi_phong=("gia_moi_phong", "median"))
# 4. tổng hợp
    .sort_values("gia_moi_phong", ascending=False)                # 5. sắp xếp
    .round(0)
)
print(ket_qua)
```

Kết quả mẫu:

```
               so_khu  gia_moi_phong
nhom_thu_nhap
Q4               3595        43480.0
Q3               4134        36199.0
Q2               4210        33094.0
Q1               4225        22843.0
```

- Bọc cả khối trong ngoặc `( ... )` để xuống dòng thoải mái, mỗi dòng một bước, có comment.
- `assign(ten=lambda d: ...)`: tạo cột mới **giữa chuỗi**; `d` là bảng ở bước hiện tại.
- Code đọc từ trên xuống như một công thức nấu ăn. Không tạo ra hàng loạt biến trung gian `df2`, `df3`, `df_tmp`.

6.23. Tốc độ: vector hóa vs apply vs vòng lặp

```
import time

t0 = time.time()
kq1 = df["gia_nha"] / df["tong_phong"]                                  # vector
hóa
```

```
t1 = time.time()
kq2 = df.apply(lambda d: d["gia_nha"] / d["tong_phong"], axis=1)          # apply
theo dòng
t2 = time.time()
kq3 = [dong["gia_nha"] / dong["tong_phong"] for _, dong in df.iterrows()] # vòng
lặp iterrows
t3 = time.time()

print(f"Vector hóa: {t1 - t0:.4f} giây")
print(f"apply     : {t2 - t1:.4f} giây")
print(f"iterrows  : {t3 - t2:.4f} giây")
```

Kết quả mẫu (thời gian trên máy bạn sẽ khác):

```
Vector hóa: 0.0001 giây
apply     : 0.0699 giây
iterrows  : 0.3219 giây
```

Cùng kết quả, chênh nhau **hàng trăm đến hàng nghìn lần**. Thứ tự ưu tiên: **vector hóa → hàm có sẵn của Pandas (`str`, `dt`, `map`, `np.where`, `np.select`) → apply → vòng lặp**.

6.24. Chuyển sang NumPy để đưa vào mô hình

```
X = df[["thu_nhap", "tuoi_nha", "phong_moi_ho"]].to_numpy()   # bảng đặc trưng →
mảng 2 chiều
y = df["gia_nha"].to_numpy()                                   # cột đích → mảng 1
chiều
print(type(X), X.shape, y.shape)
```

Kết quả mẫu:

```
<class 'numpy.ndarray'> (17000, 3) (17000,)
```

Scikit-learn nhận được cả DataFrame lẫn mảng NumPy. Quy trình chuẩn của mọi dự án ML: **Pandas để chuẩn bị dữ liệu → X, y → mô hình** (sẽ bắt đầu ở bài 2.1).

---

7. ⚠️ Lỗi thường gặp và hiểu lầm phổ biến

Lỗi 1: `KeyError` vì tên cột sai

```
xau = pd.DataFrame({"Giá ": [1, 2], "quan": ["Q1", "Q3"]})   # tên cột có dấu cách
thừa ở cuối!
try:
```

```
    xau["Giá"]
except KeyError as e:
    print("KeyError:", e)
print(xau.columns.tolist())                                    # in ra để thấy dấu
cách
xau.columns = xau.columns.str.strip()                          # sửa: bỏ khoảng
trắng thừa ở mọi tên cột
print(xau["Giá"].tolist())
```

Kết quả mẫu:

```
KeyError: 'Giá'
['Giá ', 'quan']
[1, 2]
```

- **Nhận biết:** `KeyError: 'tên_cột'`.
- **Sửa:** in `df.columns.tolist()` ra để nhìn **chính xác** từng tên. File Excel xuất ra hay có dấu cách thừa, chữ hoa, ký tự lạ. Chuẩn hóa tên cột ngay sau khi đọc file.

Lỗi 2: Dùng `and` / `or` hoặc quên ngoặc

```
try:
    df[df["thu_nhap"] > 8 and df["tuoi_nha"] < 10]
except ValueError as e:
    print("ValueError:", e)
try:
    df[df["thu_nhap"] > 8 & df["tuoi_nha"] < 10]       # quên ngoặc
except Exception as e:
    print(type(e).__name__, ":", e)
```

Kết quả mẫu:

```
ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(),
a.item(), a.any() or a.all().
TypeError : Cannot perform 'rand_' with a dtyped [float64] array and scalar of
type [bool]
```

- **Sửa:** `df[(df["thu_nhap"] > 8) & (df["tuoi_nha"] < 10)]` hoặc `df.query("thu_nhap > 8 and tuoi_nha < 10")`.

Lỗi 3: Index không khớp sau khi lọc

```
bang_con = df[df["gia_nha"] > 450000].head(3)    # 3 khu đắt; index GIỮ NHÃN GỐC,
không phải 0, 1, 2
```

```
print(bang_con.index.tolist())
thu_hang = pd.Series([1, 2, 3])                    # index 0, 1, 2
bang_con = bang_con.assign(thu=thu_hang)           # ghép theo NHÃN → không nhãn nào
khớp → toàn NaN!
print(bang_con[["gia_nha", "thu"]])
bang_con = bang_con.reset_index(drop=True).assign(thu=thu_hang)   # sửa: đánh số
lại index trước khi ghép
print(bang_con[["gia_nha", "thu"]])
```

Kết quả mẫu:

```
[215, 260, 818]
      gia_nha  thu
215  500001.0  NaN
260  500001.0  NaN
818  477600.0  NaN
    gia_nha  thu
0  500001.0    1
1  500001.0    2
2  477600.0    3
```

- **Nhận biết:** xuất hiện NaN "từ trên trời rơi xuống" sau khi gán cột hoặc cộng hai Series.
- **Sửa:** `reset_index(drop=True)` (`drop=True` để không giữ index cũ thành cột), hoặc dùng `.to_numpy()` khi bạn **cố ý** ghép theo vị trí.

Lỗi 4: Merge làm **nhân bản** dòng

```
gia_ban = pd.DataFrame({"ma_kh": ["KH1", "KH1", "KH2"],       # KH1 bị nhập TRÙNG
2 lần
                        "hang": ["Vàng", "Vàng", "Bạc"]})
kq = pd.merge(don_hang, gia_ban, on="ma_kh", how="left")
print(len(don_hang), "→", len(kq))                            # 4 đơn thành 6
dòng!
print("Tổng tiền trước:", don_hang["tien"].sum(), "| sau:", kq["tien"].sum())

try:
    pd.merge(don_hang, gia_ban, on="ma_kh", how="left", validate="many_to_one")
# bắt lỗi sớm
except Exception as e:
    print(type(e).__name__, ":", e)
```

Kết quả mẫu:

```
4 → 6
Tổng tiền trước: 465000 | sau: 645000
MergeError : Merge keys are not unique in right dataset; not a many-to-one merge
```

- **Nhận biết:** số dòng sau merge **nhiều hơn** bảng trái; tổng tiền tăng vô lý.
- **Sửa:** kiểm tra khóa bảng phải có trùng không: `gia_ban["ma_kh"].duplicated().sum()`. Thêm `validate="many_to_one"` để Pandas báo lỗi ngay nếu khóa bên phải bị trùng.

Lỗi 5: Quên gán lại kết quả

```
t = pd.DataFrame({"a": [3, 1, 2]})
t.sort_values("a")             # trả về bảng MỚI đã sắp xếp... rồi bị bỏ đi
print(t["a"].tolist())         # t vẫn như cũ!
t = t.sort_values("a")         # sửa: gán lại
print(t["a"].tolist())
```

Kết quả mẫu:

```
[3, 1, 2]
[1, 2, 3]
```

- **Nhận biết:** "Tôi đã xóa/sắp xếp/điền rồi mà sao dữ liệu không đổi?"
- **Sửa:** luôn `df = df.method(...)`. Bạn sẽ thấy tham số `inplace=True` trong tài liệu cũ; Pandas khuyên **không dùng** nữa.

Lỗi 6: Cột số bị lưu thành chữ

```
gia_chu = pd.DataFrame({"gia": ["1,200", "800", "1,500"]})    # dấu phẩy ngăn nghìn → Pandas đọc thành chữ
print(gia_chu["gia"].dtype, gia_chu["gia"].sum())              # cộng chữ = NỐI chữ!
gia_chu["gia"] = gia_chu["gia"].str.replace(",", "").astype(int)   # sửa: bỏ dấu phẩy rồi đổi kiểu
print(gia_chu["gia"].dtype, gia_chu["gia"].sum())
```

Kết quả mẫu:

```
object 1,2008001,500
int64 3500
```

- **Nhận biết:** `info()` hiện `object` ở cột lẽ ra là số; `sum()` ra chuỗi dài kỳ lạ; `describe()` không thấy cột đó.
- **Sửa:** làm sạch ký tự lạ rồi `astype`, hoặc `pd.to_numeric(cot, errors="coerce")` (ô nào không đổi được thành NaN). Khi đọc file: `pd.read_csv(..., thousands=",")`.

Lỗi 7: Sửa bảng con và nhận `SettingWithCopyWarning`

Xem hộp cảnh báo ở mục 6.11. **Quy tắc:** sửa bảng gốc → `df.loc[điều_kiện, cột] = ...`; làm việc riêng với bảng con → `.copy()`.

---

8. 🌎 Ứng dụng trong thực tế

8.1. Bán lẻ: tìm khách hàng VIP

Một shop thời trang muốn biết ai mua nhiều và lâu chưa quay lại để gửi mã giảm giá:

```
gd = pd.DataFrame({
    "ma_kh": ["KH1", "KH2", "KH1", "KH3", "KH2", "KH1", "KH3"],
    "ngay": pd.to_datetime(["2026-01-05", "2026-01-20", "2026-02-11", "2026-02-
15",
                            "2026-03-01", "2026-03-20", "2026-01-02"]),
    "tien": [500, 1200, 300, 150, 900, 700, 200],      # nghìn đồng
})
hom_nay = pd.Timestamp("2026-03-31")
rfm = gd.groupby("ma_kh").agg(
    so_ngay_chua_mua=("ngay", lambda x: (hom_nay - x.max()).days),   # Recency:
bao lâu chưa mua
    so_lan_mua=("ngay", "count"),                                    # Frequency:
mua mấy lần
    tong_chi=("tien", "sum"),                                        # Monetary:
chi bao nhiêu
)
print(rfm.sort_values("tong_chi", ascending=False))
```

Kết quả mẫu:

```
       so_ngay_chua_mua  so_lan_mua  tong_chi
ma_kh
KH2                  30           2      2100
KH1                  11           3      1500
KH3                  44           2       350
```

Đây là phân tích **RFM** (Recency – Frequency – Monetary), kỹ thuật kinh điển của marketing. KH3 chi ít và đã 44 ngày chưa quay lại: cần "đánh thức". Ở bài 2.13, bạn sẽ dùng K-Means để **tự động** chia khách hàng thành nhóm dựa trên bảng RFM này.

8.2. Ngân hàng: tỉ lệ nợ xấu theo nhóm khách

Bộ phận tín dụng có bảng hồ sơ vay với cột `thu_nhap`, `tuoi`, `no_xau` (0/1). Chỉ với `pd.cut` + `groupby("nhom_tuoi")["no_xau"].mean()`, họ biết ngay nhóm tuổi nào rủi ro cao. Đây là bước đầu tiên trước khi xây **mô hình chấm điểm tín dụng** (bài 2.4). Mẹo: trung bình của cột 0/1 chính là **tỉ lệ** (giống `mask.mean()` ở bài 1.3).

8.3. Chăm sóc khách hàng: theo dõi phàn nàn theo tuần

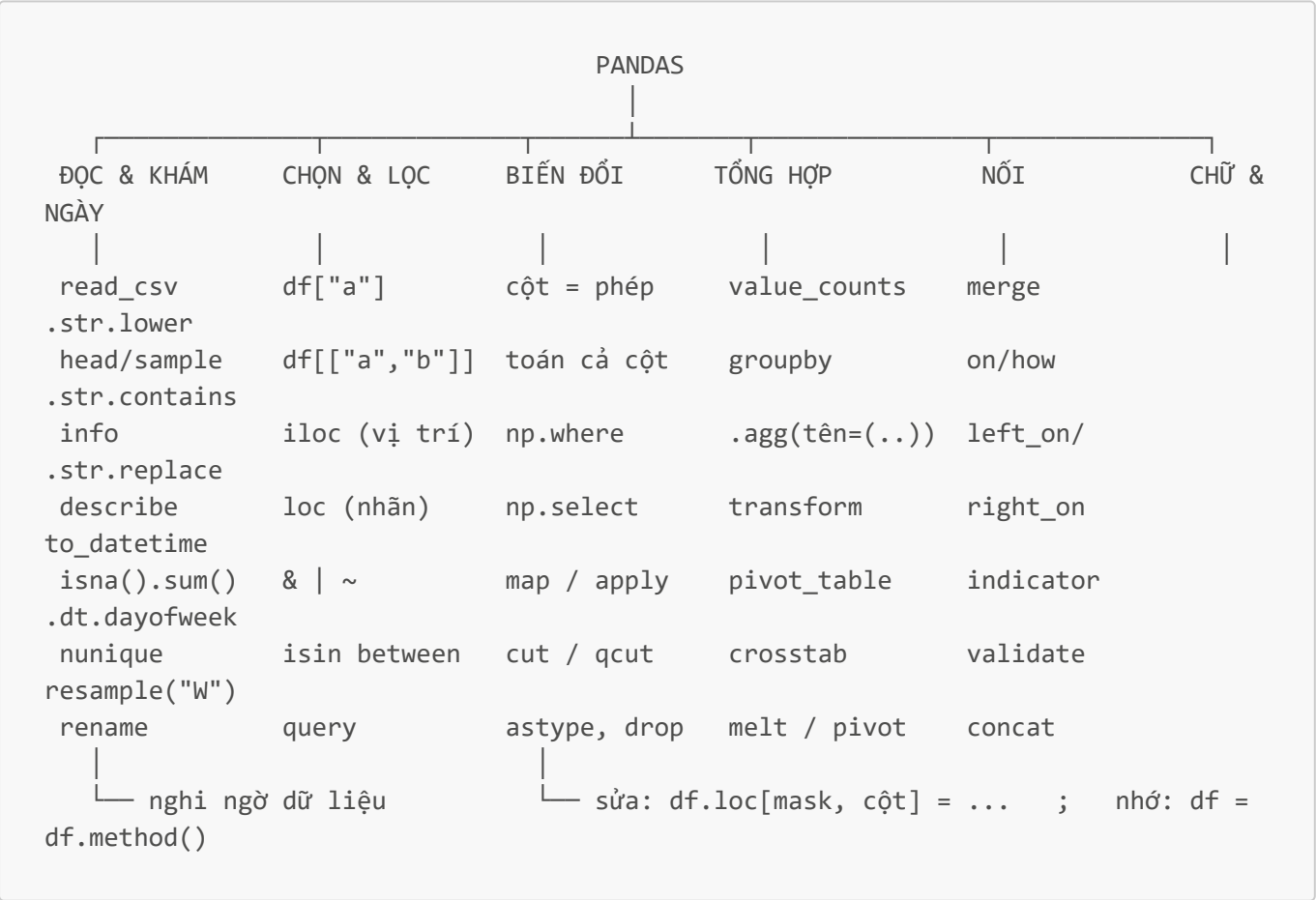
Tổng đài nhận hàng nghìn tin nhắn mỗi ngày. Kết hợp `.str.contains(["chậm", "hỏng", "hoàn tiền"])` để gắn cờ phàn nàn, `.dt` + `resample("W")` để đếm theo tuần, `pivot_table` để xem theo từng chi nhánh. Một biểu đồ đường (bài 1.5) cho thấy tuần nào số phàn nàn tăng đột biến. Ở nhánh NLP, bạn sẽ thay việc "tìm từ khóa" bằng mô hình hiểu ngữ nghĩa (bài NLP1.4, NLP3).

---

9. 📝 Tóm tắt

1. **Series** = một cột có nhãn; **DataFrame** = bảng nhiều cột, mỗi cột một kiểu; Pandas tính toán **khớp theo nhãn (index)**.
2. Đọc xong dữ liệu luôn "khám": `shape`, `head/sample`, `info()`, `describe()`, `isna().sum()`, `nunique()`. **Nghi ngờ dữ liệu trước khi tin.**
3. Chọn: `df["a"]`, `df[["a", "b"]]`, `iloc` (vị trí), `loc` (nhãn và điều kiện). Lọc: `&`, `|`, `~`, `isin`, `between`, `query`.
4. Tạo cột: phép toán cả cột, `np.where`, `np.select`, `map`, `pd.cut/qcut`; `apply` khi bất đắc dĩ. Sửa: `df.loc[điều_kiện, cột] = ...`.
5. Tổng hợp: `value_counts`, `groupby().agg()`, `transform`, `pivot_table`, `crosstab`; xoay bảng bằng `melt`/`pivot`.
6. Nối: `merge` (inner/left/right/outer) – **kiểm tra số dòng trước và sau**; `concat` để chồng bảng.
7. Chữ dùng `.str`, ngày dùng `pd.to_datetime` + `.dt` + `resample`. Method trả về bảng mới, **nhớ gán lại**.

Sơ đồ tư duy:



---

10. ✅ Trắc nghiệm

> Tự làm hết 10 câu trước, rồi mới xem mục 🔒 **Đáp án trắc nghiệm** ở cuối bài.

Mức 1 – Nhớ / Hiểu

Câu 1. Câu nào mô tả **đúng** quan hệ giữa Series và DataFrame?

- A. Series là bảng, DataFrame là một cột.
- B. Mỗi cột của DataFrame là một Series, các cột dùng chung index.
- C. DataFrame chỉ chứa được một kiểu dữ liệu cho cả bảng.
- D. Series không có index.

Câu 2. Với `df` có index mặc định 0, 1, 2..., `df.loc[0:2]` và `df.iloc[0:2]` lần lượt trả về bao nhiêu dòng?

- A. 2 và 2
- B. 3 và 3
- C. 3 và 2
- D. 2 và 3

Câu 3. Cột `gia` hiện dtype `object` trong `df.info()`. Điều đó gợi ý gì?

- A. Cột chứa toàn số nguyên lớn.
- B. Cột có thể chứa chữ hoặc ký tự lạ, cần kiểm tra và làm sạch.
- C. Cột đã được chuẩn hóa.
- D. Cột không có ô trống nào.

Câu 4. Khác biệt chính giữa `groupby(...).agg("mean")` và `groupby(...).transform("mean")` là gì?

- A. `agg` nhanh hơn nên luôn dùng `agg`.
- B. `agg` trả về mỗi nhóm một dòng; `transform` trả về kết quả cùng độ dài với bảng gốc.
- C. `transform` chỉ dùng được với cột chữ.
- D. Không khác nhau.

Mức 2 – Vận dụng

Câu 5. Đoạn code sau in ra gì?

```
s1 = pd.Series([10, 20, 30], index=["a", "b", "c"])
s2 = pd.Series([1, 2, 3], index=["b", "c", "d"])
print((s1 + s2).isna().sum())
```

- A. 0
- B. 1
- C. 2
- D. 3

Câu 6. Muốn lấy các khu có `tuoi_nha` từ 10 đến 20 **và** `thu_nhap` > 5, cách nào **sai**?

- A. `df[(df["tuoi_nha"].between(10, 20)) & (df["thu_nhap"] > 5)]`
- B. `df.query("10 <= tuoi_nha <= 20 and thu_nhap > 5")`
- C. `df[(df["tuoi_nha"] >= 10) & (df["tuoi_nha"] <= 20) & (df["thu_nhap"] > 5)]`

- D. `df[df["tuoi_nha"] >= 10 & df["tuoi_nha"] <= 20 & df["thu_nhap"] > 5]`

Câu 7. Đoạn code sau in ra gì?

```
t = pd.DataFrame({"lop": ["A", "A", "B", "B", "B"], "diem": [8, 6, 9, 7, 5]})
print(t.groupby("lop")["diem"].agg(["count", "mean"]).loc["B", "mean"])
```

- A. 7.0
- B. 3
- C. 7.5
- D. 35

Câu 8. Bảng `don` có 1.000 dòng. Sau `kq = pd.merge(don, khach, on="ma_kh", how="left")`, `len(kq)` là 1.120. Nguyên nhân khả dĩ nhất?

- A. Bảng `khach` có 120 khách chưa từng mua hàng.
- B. Bảng `khach` có mã khách bị trùng, nên một số đơn được nhân bản.
- C. `how="left"` luôn thêm dòng trống.
- D. Bảng `don` có 120 ô NaN.

Mức 3 – Phân tích (bẫy hay gặp)

Câu 9. Một bạn viết:

```
ban = df[df["quan"] == "Quận 1"]
ban["gia"] = ban["gia"] * 1.1
```

rồi kiểm tra `df` thì thấy giá Quận 1 **không** tăng, kèm cảnh báo `SettingWithCopyWarning`. Nhận định nào đúng?

- A. Pandas bị lỗi, cần cài lại.
- B. `ban` là bảng con tách từ `df`; sửa `ban` không sửa `df`. Muốn sửa `df` thì dùng `df.loc[df["quan"] == "Quận 1", "gia"] *= 1.1`.
- C. Phải thêm `inplace=True` vào phép nhân.
- D. Cảnh báo chỉ xuất hiện vì dữ liệu quá lớn.

Câu 10. Bạn tính "giá trung bình theo quận" và thấy Quận 7 có giá trung bình 12 tỷ, gấp đôi Quận 1. Kiểm tra thêm thì Quận 7 chỉ có 3 tin đăng, trong đó 1 tin giá 30 tỷ. Bạn nên làm gì **trước tiên**?

- A. Báo cáo ngay: Quận 7 đắt nhất thành phố.
- B. Xóa Quận 7 khỏi dữ liệu.
- C. Thêm số lượng (`count`) và trung vị (`median`) vào bảng `agg`, kiểm tra tin 30 tỷ có phải lỗi/ngoại lệ không, rồi mới kết luận.
- D. Đổi sang dùng `max` thay cho `mean`.

---

11. 🛠️ Thực hành

Bài 1 (Cơ bản): Báo cáo tuần của tiệm bánh

Tiệm bánh "Bánh Ngon" ghi lại các đơn hàng trong 2 tuần. Làm theo từng bước.

```
import pandas as pd
import numpy as np

don = pd.DataFrame({
    "ngay": ["2026-04-06", "2026-04-06", "2026-04-07", "2026-04-08", "2026-04-09",
"2026-04-11",
             "2026-04-12", "2026-04-13", "2026-04-14", "2026-04-15", "2026-04-18",
"2026-04-19"],
    "banh": ["Bông lan", "Croissant", "Bông lan", "Tiramisu", "Croissant",
"Tiramisu",
             "Bông lan", "Croissant", "Bông lan", "Tiramisu", "Bông lan",
"Croissant"],
    "so_luong": [10, 6, 8, 3, 7, 5, 12, 4, 9, 2, 15, 10],
    "kenh": ["Tại quán", "Online", "Online", "Tại quán", "Online", "Online",
             "Tại quán", "Tại quán", "Online", "Online", "Tại quán", "Online"],
})
bang_gia = pd.DataFrame({"banh": ["Bông lan", "Croissant", "Tiramisu"],
                         "don_gia": [35000, 28000, 55000]})

# Bước 1: In shape, info() của bảng don. Cột "ngay" đang có kiểu gì?
# TODO

# Bước 2: Đổi cột "ngay" sang kiểu ngày. (Gợi ý: pd.to_datetime)
# TODO

# Bước 3: Ghép bảng giá vào bảng đơn để có cột don_gia. Kiểm tra số dòng KHÔNG
đổi.
#         (Gợi ý: pd.merge(..., on="banh", how="left"))
# TODO

# Bước 4: Tạo cột doanh_thu = so_luong * don_gia.
# TODO

# Bước 5: Tổng doanh thu và tổng số bánh của TỪNG LOẠI bánh, sắp xếp doanh thu
giảm dần.
#         (Gợi ý: groupby("banh").agg(tong_dt=("doanh_thu", "sum"), tong_banh=
("so_luong", "sum")))
# TODO

# Bước 6: Tỉ lệ doanh thu Online vs Tại quán (%). (Gợi ý: groupby("kenh")
["doanh_thu"].sum() / tổng * 100)
# TODO

# Bước 7: Bảng pivot: dòng = loại bánh, cột = kênh, ô = tổng số lượng. Ô trống
điền 0.
#         (Gợi ý: pivot_table(..., aggfunc="sum", fill_value=0))
# TODO

# Bước 8: Doanh thu theo tuần. (Gợi ý: set_index("ngay")
```

```
["doanh_thu"].resample("W").sum())
# TODO

# Bước 9: Những đơn có số lượng lớn hơn số lượng trung bình CỦA LOẠI BÁNH ĐÓ.
#         (Gợi ý: transform("mean"))
# TODO
```

Bài 2 (Trung bình): Ai sống sót trên tàu Titanic?

Dataset: Titanic, danh sách 891 hành khách trên con tàu chìm năm 1912. Nguồn: bộ dữ liệu mẫu của thư viện Seaborn (gốc từ cuộc thi "Titanic – Machine Learning from Disaster" trên Kaggle).

```
import pandas as pd
import numpy as np
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"
tt = pd.read_csv(url)
# TODO: làm các yêu cầu dưới đây
```

Một số cột: `survived` (1 = sống sót), `pclass` (hạng vé 1, 2, 3), `sex`, `age`, `fare` (giá vé), `embark_town` (cảng lên tàu), `who` (man/woman/child), `alone` (đi một mình?), `deck` (boong tàu).

Yêu cầu:

1. Khám sức khỏe dataset: kích thước, kiểu dữ liệu, cột nào thiếu nhiều nhất (tính theo %)?
2. Tỉ lệ sống sót chung là bao nhiêu?
3. Tỉ lệ sống sót theo giới tính, theo hạng vé, và theo cả hai (pivot_table). Bạn rút ra kết luận gì?
4. Chia tuổi thành nhóm: trẻ em (0–12), thiếu niên (13–17), người lớn (18–59), người già (60+). Tỉ lệ sống sót mỗi nhóm? (Chú ý hành khách thiếu tuổi.)
5. Giá vé trung bình và trung vị theo hạng vé. Vì sao trung bình và trung vị chênh nhau nhiều ở hạng 1?
6. Người đi một mình có tỉ lệ sống sót cao hơn hay thấp hơn?
7. Top 5 vé đắt nhất: những người đó là ai (giới tính, tuổi, hạng), có sống sót không?

Gợi ý: `isna().mean()`, `groupby(...)["survived"].mean()`, `pd.cut(..., right=True)`, `pivot_table`, `nlargest`.

Bài 3 (Thử thách): Phân tích chuỗi cửa hàng từ 3 bảng

Bạn là chuyên viên phân tích của một chuỗi đồ uống. Dữ liệu gồm 3 bảng:

```
import pandas as pd
cua_hang = pd.DataFrame({"ma_ch": ["CH1", "CH2", "CH3"],
                         "thanh_pho": ["Hà Nội", "TP.HCM", "TP.HCM"]})
san_pham = pd.DataFrame({"ma_sp": ["SP1", "SP2", "SP3", "SP4"],
                         "ten_sp": ["Trà sữa", "Cà phê", "Sinh tố", "Nước ép"],
                         "gia": [45000, 35000, 50000, 40000],
                         "gia_von": [15000, 10000, 22000, 18000]})
giao_dich = pd.DataFrame({
    "ngay": pd.to_datetime(["2026-05-01", "2026-05-01", "2026-05-02", "2026-05-
```

```
08", "2026-05-09",
                              "2026-05-15", "2026-05-16", "2026-05-20", "2026-05-
22", "2026-05-29",
                              "2026-06-01", "2026-06-03", "2026-06-05", "2026-06-
12", "2026-06-20"]),
    "ma_ch": ["CH1", "CH2", "CH3", "CH1", "CH2", "CH3", "CH2", "CH1", "CH3",
"CH2",
              "CH1", "CH2", "CH3", "CH2", "CH4"],
    "ma_sp": ["SP1", "SP2", "SP1", "SP3", "SP1", "SP4", "SP2", "SP2", "SP3",
"SP1",
              "SP4", "SP3", "SP1", "SP2", "SP1"],
    "so_ly": [20, 45, 15, 10, 40, 12, 30, 25, 8, 45, 9, 14, 22, 50, 5],
})
# TODO: viết lời giải của bạn
```

Yêu cầu (không gợi ý):

1. Ghép 3 bảng thành một bảng phân tích. Phát hiện và báo cáo mọi vấn đề dữ liệu bạn gặp khi ghép.
2. Tính doanh thu và **lợi nhuận** theo cửa hàng, theo thành phố, theo sản phẩm.
3. Sản phẩm bán nhiều ly nhất có phải là sản phẩm lời nhiều nhất không? Vì sao?
4. So sánh doanh thu tháng 5 và tháng 6 của từng cửa hàng (một bảng: dòng là cửa hàng, cột là tháng, thêm cột % thay đổi).
5. Viết toàn bộ phần tính toán của câu 2 bằng **một chuỗi method chaining**.

Tiêu chí tự đánh giá:

- ☐ Phát hiện được giao dịch có mã cửa hàng không tồn tại, và giải thích được bạn xử lý nó thế nào (giữ, bỏ, hay gắn nhãn) cùng lý do.
- ☐ Số dòng bảng sau khi ghép được in ra và giải thích được.
- ☐ Lợi nhuận tính đúng = số ly × (giá − giá vốn); tổng các nhóm cộng lại khớp với tổng chung.
- ☐ Bảng câu 4 có đúng cấu trúc, % thay đổi tính đúng chiều (tháng 6 so với tháng 5).
- ☐ Code đọc được: tên biến rõ ràng, có comment, không tạo hàng loạt biến tạm `df2`, `df3`.

---

12. 🎴 Flashcard

Mặt trước	Mặt sau
`iloc` vs `loc`?	`iloc` theo vị trí số (bỏ biên phải). `loc` theo nhãn (tính cả biên phải), dùng được điều kiện.
Cách sửa giá trị an toàn?	`df.loc[điều_kiện, "cột"] = giá_trị` (một lần ngoặc).
`agg` vs `transform`?	`agg`: mỗi nhóm 1 dòng. `transform`: giữ nguyên số dòng gốc.
Sau merge cần kiểm tra gì?	Số dòng trước/sau; khóa bị trùng; dòng không khớp (`indicator=True`).
Dấu hiệu dữ liệu bị giới hạn trần?	Max là con số tròn bất thường, nhiều dòng trùng đúng giá trị đó.

---

13. 🚀 Đọc thêm

- 10 minutes to pandas – hướng dẫn nhanh chính thức.
- Pandas cheat sheet (PDF) – in ra dán cạnh màn hình.
- Comparison with spreadsheets – đối chiếu từng thao tác Excel sang Pandas.
- *Python for Data Analysis*, Wes McKinney (người tạo ra Pandas), bản đọc miễn phí tại wesmckinney.com/book.
- Khóa Kaggle Learn – Pandas – bài tập ngắn có chấm điểm tự động.

Câu hỏi gợi mở cho bài sau: Bảng `describe()` cho bạn 8 con số mỗi cột. Nhưng ở bài 1.5 bạn sẽ thấy 4 bộ dữ liệu có **cùng** trung bình, độ lệch chuẩn, tương quan mà hình dạng **khác hẳn nhau**. Và câu hỏi "nhà cũ đắt hơn vì cũ hay vì vị trí?" không thể trả lời bằng bảng số. Làm sao để **nhìn thấy** dữ liệu? Đó là bài 1.5: Matplotlib và Seaborn.

🔒 Đáp án trắc nghiệm

> Mẹo Notion: chọn tiêu đề này → `Turn into → Toggle heading 2` để ẩn đáp án.

Câu	Đáp án	Giải thích
1	**B**	Đúng như mục 4.2 và 5.1. A đảo ngược. C sai: mỗi cột một kiểu riêng (khác NumPy). D sai: Series luôn có index.
2	**C**	`loc` tính cả biên phải (0, 1, 2 → 3 dòng); `iloc` bỏ biên phải (0, 1 → 2 dòng).
3	**B**	Cột số mà dtype `object` thường do có ô chữ như `"1,200"`, `"N/A"` (Lỗi 6). A sai: số nguyên lớn vẫn là `int64`. C, D không liên quan đến dtype.
4	**B**	Mục 6.15.5. A sai: hai hàm làm việc khác nhau, không phải chuyện nhanh chậm. C sai. D sai.
5	**C**	Ghép theo nhãn: `a` chỉ có ở s1, `d` chỉ có ở s2 → 2 NaN. `b` = 20 + 1, `c` = 30 + 2. A là bẫy nếu nghĩ cộng theo vị trí.
6	**D**	Quên ngoặc: `&` được tính trước `>=`, `<=` nên gây lỗi. A, B, C đều đúng (`query` cho phép viết `10 <= tuoi_nha <= 20`).
7	**A**	Lớp B có 9, 7, 5 → trung bình 7.0. B là `count`. C, D là bẫy tính nhầm.
8	**B**	Left merge chỉ tăng số dòng khi một khóa bên trái khớp **nhiều** dòng bên phải (Lỗi 4). A sai: khách chưa mua chỉ xuất hiện với `how="right"`/`"outer"`. C sai: left giữ đúng số dòng nếu khóa phải không trùng. D không liên quan.
9	**B**	Mục 6.11 và Lỗi 7. A, D sai. C sai: phép gán cột không có `inplace`, và `inplace` cũng không giải quyết vấn đề bảng con.
10	**C**	Trung bình rất nhạy với ngoại lệ khi nhóm có ít dòng. Luôn xem `count` và `median` cùng `mean`. A kết luận vội. B xóa dữ liệu thật vô căn cứ. D càng nhạy với ngoại lệ hơn.

🔒 Lời giải thực hành

> Mẹo Notion: chọn tiêu đề này → `Turn into → Toggle heading 2`.

Lời giải Bài 1

```
import pandas as pd
import numpy as np

don = pd.DataFrame({
    "ngay": ["2026-04-06", "2026-04-06", "2026-04-07", "2026-04-08", "2026-04-09",
"2026-04-11",
             "2026-04-12", "2026-04-13", "2026-04-14", "2026-04-15", "2026-04-18",
"2026-04-19"],
    "banh": ["Bông lan", "Croissant", "Bông lan", "Tiramisu", "Croissant",
"Tiramisu",
             "Bông lan", "Croissant", "Bông lan", "Tiramisu", "Bông lan",
"Croissant"],
    "so_luong": [10, 6, 8, 3, 7, 5, 12, 4, 9, 2, 15, 10],
    "kenh": ["Tại quán", "Online", "Online", "Tại quán", "Online", "Online",
             "Tại quán", "Tại quán", "Online", "Online", "Tại quán", "Online"],
})
bang_gia = pd.DataFrame({"banh": ["Bông lan", "Croissant", "Tiramisu"],
                         "don_gia": [35000, 28000, 55000]})

print(don.shape); don.info()                                          # 1: "ngay"
đang là object (chữ)
don["ngay"] = pd.to_datetime(don["ngay"])                             # 2

truoc = len(don)                                                      # 3
don = pd.merge(don, bang_gia, on="banh", how="left")
print(truoc, len(don))

don["doanh_thu"] = don["so_luong"] * don["don_gia"]                   # 4

print(don.groupby("banh").agg(tong_dt=("doanh_thu", "sum"),           # 5
                              tong_banh=("so_luong", "sum"))
         .sort_values("tong_dt", ascending=False))

theo_kenh = don.groupby("kenh")["doanh_thu"].sum()                    # 6
print((theo_kenh / theo_kenh.sum() * 100).round(1))

print(don.pivot_table(values="so_luong", index="banh", columns="kenh",   # 7
                      aggfunc="sum", fill_value=0))

print(don.set_index("ngay")["doanh_thu"].resample("W").sum())         # 8

tb_loai = don.groupby("banh")["so_luong"].transform("mean")           # 9
print(don.loc[don["so_luong"] > tb_loai, ["ngay", "banh", "so_luong"]])
```

Kết quả mẫu:

```
(12, 4)
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 12 entries, 0 to 11
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   ngay      12 non-null     object
 1   banh      12 non-null     object
 2   so_luong  12 non-null     int64 
 3   kenh      12 non-null     object
dtypes: int64(1), object(3)
memory usage: 516.0+ bytes
12 12
           tong_dt  tong_banh
banh                         
Bông lan   1890000         54
Croissant   756000         27
Tiramisu    550000         10
kenh
Online      50.8
Tại quán    49.2
Name: doanh_thu, dtype: float64
kenh       Online  Tại quán
banh                       
Bông lan       17        37
Croissant      23         4
Tiramisu        7         3
ngay
2026-04-12    1854000
2026-04-19    1342000
Freq: W-SUN, Name: doanh_thu, dtype: int64
         ngay       banh  so_luong
4  2026-04-09  Croissant         7
5  2026-04-11   Tiramisu         5
6  2026-04-12   Bông lan        12
10 2026-04-18   Bông lan        15
11 2026-04-19  Croissant        10
```

Lời giải Bài 2

```python
import pandas as pd
import numpy as np
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"
tt = pd.read_csv(url)

# 1. Khám sức khỏe
print(tt.shape)
print((tt.isna().mean() * 100).round(1).sort_values(ascending=False).head(4))   # % thiếu mỗi cột

# 2. Tỉ lệ sống sót chung
print(round(tt["survived"].mean(), 3))
```

```
# 3. Theo giới tính, hạng vé, và cả hai
print(tt.groupby("sex")["survived"].mean().round(3))
print(tt.groupby("pclass")["survived"].mean().round(3))
print(tt.pivot_table(values="survived", index="sex", columns="pclass",
aggfunc="mean").round(2))

# 4. Nhóm tuổi. Người thiếu tuổi sẽ có nhóm NaN và bị groupby bỏ qua → đếm riêng
tt["nhom_tuoi"] = pd.cut(tt["age"], bins=[0, 12, 17, 59, 120],
                         labels=["Trẻ em", "Thiếu niên", "Người lớn", "Người
già"])
print(tt.groupby("nhom_tuoi", observed=True)["survived"].agg(["count",
"mean"]).round(3))
print("Thiếu tuổi:", tt["age"].isna().sum(), "người")

# 5. Giá vé theo hạng
print(tt.groupby("pclass")["fare"].agg(["mean", "median", "max"]).round(1))

# 6. Đi một mình
print(tt.groupby("alone")["survived"].mean().round(3))

# 7. Top 5 vé đắt nhất
print(tt.nlargest(5, "fare")[["sex", "age", "pclass", "fare", "survived"]])
```

Kết quả mẫu:

```
(891, 15)
deck           77.2
age            19.9
embarked        0.2
embark_town     0.2
dtype: float64
0.384
sex
female    0.742
male      0.189
Name: survived, dtype: float64
pclass
1    0.630
2    0.473
3    0.242
Name: survived, dtype: float64
pclass     1     2     3
sex
female  0.97  0.92  0.50
male    0.37  0.16  0.14
            count   mean
nhom_tuoi
Trẻ em         69  0.580
Thiếu niên     44  0.477
Người lớn     575  0.386
Người già      26  0.269
Thiếu tuổi: 177 người
```

```
        mean  median    max
pclass
1       84.2    60.3  512.3
2       20.7    14.2   73.5
3       13.7     8.0   69.6
alone
False    0.506
True     0.304
Name: survived, dtype: float64
        sex   age  pclass      fare  survived
258  female  35.0       1  512.3292         1
679    male  36.0       1  512.3292         1
737    male  35.0       1  512.3292         1
27     male  19.0       1  263.0000         0
88   female  23.0       1  263.0000         1
```

Nhận xét:

- `deck` thiếu hơn 77% → gần như không dùng được; `age` thiếu gần 20% → cần xử lý (bài 1.6).
- Chỉ 38% sống sót. Nữ sống sót 74%, nam 19%. Hạng 1 sống sót 63%, hạng 3 chỉ 24%. Bảng pivot cho thấy **nữ hạng 1, 2** gần như đều sống sót, còn **nam hạng 3** chỉ khoảng 14%. Đúng nguyên tắc "phụ nữ và trẻ em trước" cùng sự bất công về vị trí khoang tàu.
- Trẻ em có tỉ lệ sống sót cao nhất trong các nhóm tuổi.
- Hạng 1: trung bình 84 nhưng trung vị 60, vì có vài vé cực đắt (512) kéo trung bình lên. Trung vị **ít bị ảnh hưởng bởi giá trị cực đoan** (bài 1.7).
- Đi một mình sống sót **thấp hơn** (30% so với 51%). Có thể vì người đi một mình phần lớn là nam giới trẻ, hạng 3. Lại là "tương quan không phải nhân quả" (bài 1.5).

Lời giải Bài 3

```
import pandas as pd
cua_hang = pd.DataFrame({"ma_ch": ["CH1", "CH2", "CH3"],
                         "thanh_pho": ["Hà Nội", "TP.HCM", "TP.HCM"]})
san_pham = pd.DataFrame({"ma_sp": ["SP1", "SP2", "SP3", "SP4"],
                         "ten_sp": ["Trà sữa", "Cà phê", "Sinh tố", "Nước ép"],
                         "gia": [45000, 35000, 50000, 40000],
                         "gia_von": [15000, 10000, 22000, 18000]})
giao_dich = pd.DataFrame({
    "ngay": pd.to_datetime(["2026-05-01", "2026-05-01", "2026-05-02", "2026-05-
08", "2026-05-09",
                            "2026-05-15", "2026-05-16", "2026-05-20", "2026-05-
22", "2026-05-29",
                            "2026-06-01", "2026-06-03", "2026-06-05", "2026-06-
12", "2026-06-20"]),
    "ma_ch": ["CH1", "CH2", "CH3", "CH1", "CH2", "CH3", "CH2", "CH1", "CH3",
"CH2",
              "CH1", "CH2", "CH3", "CH2", "CH4"],
    "ma_sp": ["SP1", "SP2", "SP1", "SP3", "SP1", "SP4", "SP2", "SP2", "SP3",
"SP1",
              "SP4", "SP3", "SP1", "SP2", "SP1"],
```

```
    "so_ly": [20, 45, 15, 10, 40, 12, 30, 25, 8, 45, 9, 14, 22, 50, 5],
})

# 1. Ghép, có kiểm tra
print("Khóa trùng:", cua_hang["ma_ch"].duplicated().sum(),
san_pham["ma_sp"].duplicated().sum())
bang = (giao_dich
        .merge(san_pham, on="ma_sp", how="left", validate="many_to_one")
        .merge(cua_hang, on="ma_ch", how="left", validate="many_to_one",
indicator=True))
print(len(giao_dich), "→", len(bang))
print(bang.loc[bang["_merge"] == "left_only", ["ngay", "ma_ch", "ma_sp",
"so_ly"]])
# CH4 không có trong danh sách cửa hàng: GIỮ giao dịch (tiền là thật), gắn nhãn
"Chưa rõ" để báo bộ phận dữ liệu
bang["thanh_pho"] = bang["thanh_pho"].fillna("Chưa rõ")
bang = bang.drop(columns="_merge")

bang["doanh_thu"] = bang["so_ly"] * bang["gia"]
bang["loi_nhuan"] = bang["so_ly"] * (bang["gia"] - bang["gia_von"])

# 2. Doanh thu, lợi nhuận theo 3 chiều
for cot in ["ma_ch", "thanh_pho", "ten_sp"]:
    print(bang.groupby(cot)[["doanh_thu",
"loi_nhuan"]].sum().sort_values("loi_nhuan", ascending=False))
print("Kiểm tra tổng:", bang["loi_nhuan"].sum(),
      bang.groupby("ten_sp")["loi_nhuan"].sum().sum())

# 3. Bán nhiều ly nhất vs lời nhiều nhất
sp = bang.groupby("ten_sp").agg(so_ly=("so_ly", "sum"), loi_nhuan=("loi_nhuan",
"sum"))
sp["lai_moi_ly"] = sp["loi_nhuan"] / sp["so_ly"]
print(sp.sort_values("so_ly", ascending=False))

# 4. Tháng 5 vs tháng 6 theo cửa hàng
bang["thang"] = bang["ngay"].dt.month
so_sanh = bang.pivot_table(values="doanh_thu", index="ma_ch", columns="thang",
                           aggfunc="sum", fill_value=0)
so_sanh["thay_doi_%"] = ((so_sanh[6] - so_sanh[5]) / so_sanh[5] * 100).round(1)
print(so_sanh)

# 5. Câu 2 bằng một chuỗi method chaining (theo sản phẩm)
print(
    giao_dich
    .merge(san_pham, on="ma_sp", how="left")
    .assign(doanh_thu=lambda d: d["so_ly"] * d["gia"],
            loi_nhuan=lambda d: d["so_ly"] * (d["gia"] - d["gia_von"]))
    .groupby("ten_sp")[["doanh_thu", "loi_nhuan"]]
    .sum()
    .sort_values("loi_nhuan", ascending=False)
)
```

Kết quả mẫu:

```
Khóa trùng: 0 0
15 → 15
         ngay ma_ch ma_sp  so_ly
14 2026-06-20   CH4   SP1      5
       doanh_thu  loi_nhuan
ma_ch
CH2      8900000    6067000
CH1      2635000    1703000
CH3      2545000    1598000
CH4       225000     150000
           doanh_thu  loi_nhuan
thanh_pho
TP.HCM      11445000    7665000
Hà Nội       2635000    1703000
Chưa rõ       225000     150000
         doanh_thu  loi_nhuan
ten_sp
Trà sữa    6615000    4410000
Cà phê     5250000    3750000
Sinh tố    1600000     896000
Nước ép     840000     462000
Kiểm tra tổng: 9518000 9518000
         so_ly  loi_nhuan  lai_moi_ly
ten_sp
Cà phê     150    3750000     25000.0
Trà sữa    147    4410000     30000.0
Sinh tố     32     896000     28000.0
Nước ép     21     462000     22000.0
thang        5        6  thay_doi_%
ma_ch
CH1    2275000   360000       -84.2
CH2    6450000  2450000       -62.0
CH3    1555000   990000       -36.3
CH4          0   225000         inf
         doanh_thu  loi_nhuan
ten_sp
Trà sữa    6615000    4410000
Cà phê     5250000    3750000
Sinh tố    1600000     896000
Nước ép     840000     462000
```

Giải thích:

- Ghép xong vẫn 15 dòng (không nhân bản, nhờ `validate`). Có 1 giao dịch của **CH4**, cửa hàng không có trong danh mục: có thể là cửa hàng mới mở chưa được cập nhật. Không nên xóa vì doanh thu là thật; gắn nhãn "Chưa rõ" và báo lại.
- `inf` ở cột `thay_doi_%` của CH4: tháng 5 bằng 0 nên chia cho 0. Cần ghi chú "cửa hàng mới" thay vì báo "tăng vô hạn".
- Câu 3: **Cà phê** bán nhiều ly nhất (150 ly) nhưng **Trà sữa** mới lời nhiều nhất, vì lãi mỗi ly của trà sữa là 30.000đ so với 25.000đ của cà phê. Bán nhiều chưa chắc lời nhiều, một bài học kinh doanh quan trọng.